

Progettazione Metodologica di una Pipeline di Machine Learning per Sistemi di Rilevamento Intrusioni basata sul Dataset UNSW-NB15

Roberto Giuseppe Scarpa^{1,*,\dagger}, Marco Renella^{1,*,\dagger}

¹Dipartimento di Informatica, Università degli Studi di Salerno, Fisciano, Italia

*Corresponding authors: r.scarpa16@studenti.unisa.it; m.renella1@studenti.unisa.it

^{\dagger}These authors contributed equally to this work.

La continua evoluzione delle minacce informatiche rende essenziale lo sviluppo di Sistemi di Rilevamento delle Intrusioni di Rete (NIDS) basati su tecniche di Machine Learning. Questo articolo presenta una pipeline metodologica completa per l'analisi e la classificazione del traffico di rete mediante il dataset UNSW-NB15, un benchmark moderno che supera le limitazioni dei dataset legacy incorporando minacce contemporanee e traffico benigno sintetizzato con elevato realismo. Il problema viene affrontato configurando un task di classificazione supervisionata binaria per l'identificazione del traffico malevolo. Viene descritta una pipeline di preprocessing per la rimozione dei duplicati, la prevenzione del data leakage e la trasformazione delle feature numeriche e categoriche, seguita dall'ottimizzazione e valutazione di diversi algoritmi di Machine Learning mediante Grid Search e validazione incrociata stratificata. Le prestazioni vengono analizzate attraverso metriche complementari, tra cui Accuracy, Precision, Recall, F1-score, ROC-AUC e PR-AUC. Infine, sul modello con le migliori prestazioni viene applicata una procedura di feature selection basata sull'importanza delle variabili, dimostrando come sia possibile ridurre significativamente la dimensionalità preservando le capacità predittive del classificatore e migliorandone l'efficienza computazionale.

Introduzione

La sicurezza delle infrastrutture informatiche moderne dipende criticamente dalla capacità di identificare e mitigare tempestivamente le attività malevole all'interno di reti complesse. In questo contesto, i Sistemi di Rilevamento delle Intrusioni di Rete (NIDS - Network Intrusion Detection Systems) rappresentano la prima linea di difesa, analizzando il traffico in transito per distinguere le connessioni legittime dai tentativi di intrusione. Storicamente, i sistemi basati su firma (signature-based) hanno dominato l'industria; tuttavia, essi mostrano limiti evidenti nell'identificazione di attacchi inediti, noti come minacce "Zero-Day".

L'adozione del Machine Learning (ML) promette di superare tali limitazioni attraverso il rilevamento di anomalie (anomaly detection) e la classificazione comportamentale. Per decenni, la ricerca si è basata su dataset obsoleti, quali il KDD99 e il suo successore NSL-KDD, i quali non riflettono più la topologia del traffico moderno né la complessità dei vettori di attacco odierni. Per ovviare a questo problema, è stato introdotto il dataset UNSW-NB15 (1), concepito per catturare un'ampia gamma di attacchi contemporanei e traffico normale realistico.

Lo sviluppo di modelli efficaci su tali dati richiede tuttavia un'attenta analisi esplorativa e una rigorosa preparazione delle variabili. L'elevata dimensionalità, le distribuzioni non gaussiane dei parametri di rete e la presenza di *concept drift* richiedono metodologie di ingegneria delle feature specifiche per il dominio della cybersecurity. L'obiettivo di questo studio è pertanto definire una pipeline metodologica completa per la preparazione del dataset UNSW-NB15, gettando le basi per l'addestramento di classificatori robusti in grado di operare in ambienti di rete reali.

Stato dell'Arte

Evoluzione dei Sistemi di Intrusion Detection

La letteratura scientifica riguardante i Sistemi di Rilevamento delle Intrusioni di Rete (NIDS) ha subito una profonda trasformazione nell'ultimo decennio, spinta dalla continua e rapida evoluzione delle minacce informatiche. Storicamente, la difesa perimetrale delle reti si è basata su approcci *Signature-Based*, i quali operano confrontando il traffico in transito con un database di firme virali o pattern di attacco noti. Sebbene tali sistemi garantiscano un'elevata precisione e un basso

tasso di falsi positivi per le minacce consolidate, essi risultano intrinsecamente inefficaci contro le vulnerabilità *Zero-Day* e le varianti polimorfiche degli attacchi. Per superare questa limitazione fondamentale, la comunità accademica si è orientata verso architetture *Anomaly-Based*, progettate per modellare il comportamento fisiologico della rete e segnalare qualsiasi deviazione statisticamente significativa come una potenziale compromissione. È in questo contesto che l'adozione degli algoritmi di Machine Learning ha rivoluzionato il campo, fornendo gli strumenti matematici necessari per inferire pattern complessi da volumi massivi di traffico di rete, automatizzando la creazione di profili comportamentali dinamici che sfuggono alla definizione manuale di regole euristiche.

L'Evoluzione dei Dataset di Riferimento

Lo sviluppo e la validazione dei modelli di Machine Learning per i NIDS sono intrinsecamente legati alla qualità dei dati utilizzati per il loro addestramento. Alla fine degli anni Novanta, il dataset DARPA 1998 e il suo derivato KDD Cup 99 hanno rappresentato i primi benchmark standardizzati per la valutazione degli IDS. Tuttavia, la comunità scientifica ha rapidamente identificato gravi carenze strutturali in questi archivi, tra cui un'enorme mole di record duplicati che induceva i classificatori a sovrastimare le prestazioni sulle classi maggioritarie. La successiva introduzione di NSL-KDD ha parzialmente mitigato il problema della ridondanza, fornendo un set di dati più bilanciato.

Nonostante tali correzioni, gran parte degli sforzi di ricerca iniziali si è concentrata sull'applicazione di modelli su questi dataset della famiglia KDD, riportando tassi di rilevamento irrealisticamente alti. Numerosi autori hanno evidenziato come le performance quasi perfette ottenute su tali dataset non si traducevano in un'effettiva efficacia operativa. Tale discrepanza è attribuibile all'incapacità di questi archivi storici di rappresentare fedelmente la topologia e la crittografia del traffico di rete moderno, ormai dominato da protocolli cifrati (come TLS 1.3), da architetture cloud-based e da attacchi di tipo *low-and-slow*, progettati specificamente per eludere le soglie volumetriche di rilevamento.

Per far fronte all'obsolescenza dei benchmark storici, la pubblicazione del dataset UNSW-NB15 (1) ha fornito un nuovo standard per la comunità. Generato in un ambiente ibrido che combina traffico sintetico malevolo e flussi benigni reali, esso include tracce di attacchi contemporanei

suddivisi in nove famiglie distinte, offrendo un livello di realismo senza precedenti. Sebbene negli anni successivi siano stati proposti ulteriori dataset di alta qualità, come CICIDS2017 e CSE-CIC-IDS2018, il dataset UNSW-NB15 rimane uno dei benchmark più rigorosi e complessi, grazie all'inclusione di feature ingegnerizzate che catturano le dinamiche temporali e comportamentali dei flussi, rendendolo il banco di prova ideale per la valutazione di architetture di rilevamento avanzate.

Tuttavia, l'impiego dei dataset più moderni richiede estrema cautela, poiché non sono immuni da profonde criticità strutturali. Nel caso del diffuso CICIDS2017, ad esempio, analisi forensi hanno rivelato che lo strumento di estrazione CICFlowMeter troncava arbitrariamente i flussi TCP dopo un singolo pacchetto FIN, generando record frammentati e duplicati. A ciò si aggiungono errori di etichettatura: oltre il 79% della classe "Background Attack Traffic" proveniva da IP non malevoli, mentre reali payload di attacco venivano classificati come benigni.

Addestrando i modelli su dati così viziati, si incorre nel fenomeno delle correlazioni spurie: gli algoritmi memorizzano puri artefatti di rete (come IP o MAC address specifici della simulazione) anziché la semantica generalizzata dell'attacco. Di conseguenza, accuratèzze apparenti del 99% su CICIDS2017 crollano quasi a zero se applicate a topologie leggermente modificate. Per ovviare a tali distorsioni e garantire che il modello apprenda i veri pattern di intrusione, i ricercatori si sono visti costretti a correggere manualmente i dati, introducendo ad esempio l'etichetta "Attempted" per isolare i payload falliti e separare così l'intento malevolo dal suo reale impatto sulla rete.

Sviluppo degli Algoritmi di Apprendimento Automatico

Parallelamente all'evoluzione dei dati, si è assistito a una transizione paradigmatica negli algoritmi impiegati per la classificazione del traffico. Inizialmente, la ricerca ha esplorato modelli statistici e algoritmi di Machine Learning classico, quali Support Vector Machines (SVM), Naive Bayes e K-Nearest Neighbors (KNN). Tali approcci si sono rivelati computazionalmente leggeri e facilmente interpretabili, ma hanno mostrato limiti nel gestire l'elevata dimensionalità dello spazio delle feature e le relazioni non lineari intrinseche ai dati di rete.

Per superare la rigidità dei classificatori singoli, i lavori più recenti hanno esplorato estesamente le architetture ad *ensemble*. Algoritmi basati su alberi decisionali multipli, in particolare Random Forest ed Extreme Gradient Boosting (XGBoost), sono diventati lo standard *de facto* nella letteratura

applicata ai NIDS. Il loro successo deriva dalla peculiare metodologia strutturale con cui operano: a differenza di altri approcci, gli algoritmi basati su alberi costruiscono modelli costanti a tratti partizionando ricorsivamente lo spazio delle feature tramite divisioni (split) ortogonali allineate agli assi.

Questa architettura garantisce agli *ensemble* di alberi diversi vantaggi distintivi. In primo luogo, la divisione ortogonale permette di catturare in modo naturale interazioni complesse e non lineari tra le feature, senza la necessità di ricorrere a una laboriosa ingegnerizzazione manuale delle caratteristiche polinomiali. Ad esempio, l'implicazione di sicurezza di un conteggio di byte straordinariamente elevato differisce in modo fondamentale a seconda che il traffico utilizzi il protocollo UDP (indicando un potenziale attacco DDoS di amplificazione) o il TCP sulla porta 443 (indicando un normale download di file sicuro). Gli alberi decisionali incapsulano perfettamente questa logica condizionale attraverso le divisioni consecutive dei nodi.

Oltre a ciò, queste architetture mostrano una naturale resistenza alla multicollinearità, una spiccata capacità di gestire feature eterogenee (numeriche e categoriche) senza complesse trasformazioni spaziali e una robustezza intrinseca contro l'overfitting. Tali modelli riescono quindi a mappare efficacemente le complesse relazioni presenti nelle 36 feature di dataset come l'UNSW-NB15, offrendo al contempo un elevato grado di interpretabilità attraverso l'analisi dell'importanza delle variabili (*Feature Importance*).

Infine, ben oltre la loro superiorità predittiva, la natura discreta degli *ensemble* di alberi fornisce una difesa intrinseca contro specifiche classi di attacchi legati all'Adversarial Machine Learning. Gli attacchi di evasione basati sul gradiente, come il Fast Gradient Sign Method (FGSM) e il Projected Gradient Descent (PGD), si fondano sulla differenziazione della funzione di perdita del modello bersaglio per calcolare le perturbazioni ottimali da applicare all'input. Poiché gli *ensemble* di alberi si basano su soglie di divisione non differenziabili, risultano strutturalmente opachi ai classici avversari basati sul gradiente. Questa caratteristica costringe gli attaccanti a impiegare tecniche di ottimizzazione *derivative-free* (prive di derivate), che risultano molto più onerose dal punto di vista computazionale per la generazione di pacchetti avversari.

L’Impatto del Deep Learning nei NIDS

Negli ultimi anni, la necessità di estrarre rappresentazioni latenti di alto livello dai flussi di rete ha spinto la ricerca verso l’adozione del Deep Learning. Modelli non supervisionati o semi-supervisionati, come gli Autoencoder, sono stati ampiamente studiati per la loro capacità di apprendere la varietà fisiologica del traffico normale, segnalando gli attacchi come errori di ricostruzione. Per catturare le dipendenze sequenziali e temporali dei pacchetti, la letteratura ha proposto l’impiego di Long Short-Term Memory networks (LSTM), dimostrando una notevole efficacia nell’identificare vettori di attacco distribuiti nel tempo, come le campagne di *Reconnaissance* o i tentativi di *Brute Force*.

Più recentemente, l’applicazione di Reti Neurali Convoluzionali (CNN) sui *payload* trasformati in matrici bidimensionali e l’introduzione di modelli basati sul meccanismo di Attenzione (Transformers) hanno stabilito nuovi primati accademici nell’Anomaly Detection spaziale e sequenziale. Tuttavia, nonostante le eccezionali capacità di astrazione, l’implementazione del Deep Learning in ambienti SOC (Security Operations Center) reali incontra forti resistenze. L’enorme costo computazionale in fase di inferenza e, soprattutto, la natura *black-box* di tali architetture minano la validità dell’allarme: un analista di sicurezza necessita di motivazioni trasparenti per intraprendere azioni di mitigazione, una caratteristica che le reti neurali profonde faticano a fornire rispetto agli algoritmi ad albero.

Problematiche Metodologiche Aperte nella Letteratura

Nonostante l’abbondanza di modelli matematicamente sofisticati, un’analisi critica della letteratura rivela che molte delle soluzioni proposte trascurano problematiche metodologiche fondamentali, rendendo le prestazioni dichiarate inapplicabili nel mondo reale.

La prima criticità dominante è il *Class Imbalance*. Nei dataset di rete realistici, come in produzione, il traffico benigno costituisce la quasi totalità dei flussi, relegando gli attacchi a una minuscola frazione statistica. Modelli addestrati su distribuzioni originarie tendono a sviluppare un bias estremo verso la classe maggioritaria, ignorando minacce rare e silenziose (es. *Worms* o *Shellcode*). Sebbene la letteratura suggerisca l’uso di tecniche di oversampling come SMOTE o ADASYN per bilanciare artificialmente lo spazio di addestramento, l’applicazione indiscriminata

di tali metodi su dati di rete rischia di interpolare vettori che violano le specifiche dei protocolli TCP/IP, generando traffico sintetico irrealistico. Alternative più rigorose, come l'assegnazione di pesi di classe (*Class Weighting*) o l'undersampling selettivo, sono spesso trascurate o implementate in modo sub-ottimale.

Un secondo difetto sistematico è rappresentato dal *Data Leakage* (fuga di dati). Numerosi studi scientifici includono nella fase di addestramento feature identificative, quali indirizzi IP, timestamp, o identificatori di flusso (es. la colonna *id*). In queste condizioni, il modello non apprende la firma comportamentale dell'attacco, bensì memorizza semplicemente la topologia della rete in cui il dataset è stato catturato. Questo errore procedurale conduce a validazioni in cui il classificatore mostra prestazioni perfette, ma che collasserebbe immediatamente se esposto a una topologia di rete differente, dimostrando un'assoluta mancanza di capacità di generalizzazione. A ciò si lega il problema del bias spazio-temporale: addestrare e valutare un modello su dati cronologici tramite una validazione incrociata casuale (*k-fold*) permette all'algoritmo di usare un contesto "futuro" per classificare eventi "passati", un'impossibilità logica nel mondo reale. Inoltre, i ricercatori spesso ignorano profondi artefatti di estrazione: come dimostrato da Engelen et al., oltre il 25% dei flussi nel dataset CICIDS2017 risultava viziato (es. flussi TCP troncati prematuramente), spingendo i modelli a sfruttare tali correlazioni spurie anziché la reale semantica dell'attacco.

A ciò si aggiunge la sfida del *Concept Drift*. La natura intrinsecamente antagonistica della cybersecurity implica che i vettori di attacco mutino continuamente per eludere le difese. Il drift si manifesta sia come *Covariate Shift* (cambiamenti nelle abitudini del traffico benigno, es. un passaggio massivo al lavoro da remoto), sia come *Concept Shift* (alterazione tra feature ed etichette dovuta a nuovi attacchi *zero-day*). I modelli addestrati su distribuzioni statiche subiscono un inesorabile degrado delle prestazioni. Sebbene la maggior parte della letteratura si limiti a valutazioni offline su partizioni statiche, la ricerca si orienta verso approcci di apprendimento incrementale (*Online Learning*) e continuo (*Continual Learning*). Attraverso rilevatori di drift (es. ADWIN o il test Page-Hinkley), questi framework aggiornano parzialmente il modello per accumulare conoscenza, cercando di prevenire l'*oblio catastrofico* (tendenza della rete neurale a sovrascrivere i pesi e dimenticare i vecchi attacchi). Tale limitazione enfatizza l'importanza del *Feature Engineering*: solo un preprocessing rigoroso, con attenta normalizzazione e la selezione di feature comportamentali invarianti rispetto alle mutazioni del payload, può garantire una robustezza duratura.

Infine, la lacuna più evidente risiede nella valutazione empirica dei modelli. Una vasta porzione di articoli scientifici ottimizza e riporta come metrica principale l'Accuratezza Globale (Accuracy). In presenza del suddetto sbilanciamento delle classi, l'ottimizzazione dell'Accuracy risulta ingannevole a causa del Paradosso dell'Accuratezza: un classificatore *dummy* che predice sempre traffico normale otterrebbe un punteggio superiore al 99%, fallendo però lo scopo stesso per cui è stato concepito. Allo stesso modo, anche la comunemente impiegata ROC-AUC mostra limiti matematici a causa di un marcato effetto tetto (*ceiling effect*): poiché il tasso di falsi positivi è definito come $FPR = \frac{FP}{FP+TN}$, gli enormi volumi di pacchetti benigni al denominatore mascherano le quantità assolute di falsi positivi, gonfiando artificialmente il punteggio. La letteratura più rigorosa richiede la massimizzazione della *Recall* (Sensibilità) per minimizzare i falsi negativi (intrusioni non rilevate), bilanciata dalla *Precision* per contenere i falsi positivi che generano *Alert Fatigue* negli analisti operativi. Di conseguenza, il F1-Score e, soprattutto, l'Area Sotto la Curva Precision-Recall (PR-AUC) rappresentano gli unici indicatori matematicamente onesti per valutare l'efficacia di un NIDS.

Gap Scientifico e Posizionamento del Presente Lavoro

Alla luce della rassegna condotta, emerge come il problema dell'intrusion detection non dipenda esclusivamente dalla scelta dell'algoritmo predittivo, ma anche dalla qualità del protocollo sperimentale adottato. In molti lavori, prestazioni elevate possono derivare da scelte metodologiche poco controllate, come l'utilizzo improprio del test set, la presenza di feature informative in modo non realistico, la mancata gestione di duplicati o ridondanze, oppure l'impiego di metriche non adeguate al costo operativo degli errori in un contesto NIDS.

Il presente lavoro si colloca in questa prospettiva metodologica. L'obiettivo non è proporre una nuova architettura complessa, ma costruire una pipeline di Data Science chiara, riproducibile e coerente con le caratteristiche del dataset UNSW-NB15.

Attraverso una rigorosa definizione delle fasi di data exploration e preprocessing applicate al dataset UNSW-NB15, e l'utilizzo di metriche di valutazione più informative per il dominio della sicurezza, il lavoro fornisce una pipeline metodologica trasparente, riproducibile e ottimizzata per l'integrazione reale nei moderni scenari di Incident Response.

Metodi e Materiali

Caratteristiche del Dataset e Analisi delle Criticità

Il dataset UNSW-NB15 è stato generato presso il Cyber Range Lab dell'UNSW Canberra mediante lo strumento IXIA PerfectStorm, con l'obiettivo di produrre un ambiente ibrido composto da traffico normale realistico e comportamenti malevoli sintetici contemporanei. Il traffico grezzo è stato catturato tramite `tcpdump`, mentre strumenti come Argus e Bro-IDS sono stati impiegati per estrarre feature descrittive dei flussi di rete. Nella sua formulazione originale, il dataset comprende oltre due milioni di record distribuiti in più file CSV e include nove famiglie di attacco: Fuzzers, Analysis, Backdoors, DoS, Exploits, Generic, Reconnaissance, Shellcode e Worms (1).

Nel presente progetto è stata utilizzata la versione in formato Parquet disponibile su Kaggle, derivata dagli split ufficiali `UNSW_NB15_training-set` e `UNSW_NB15_testing-set`. Tale versione mantiene la partizione standard del benchmark, composta da 175.341 record nel training set e 82.332 record nel test set. Entrambe le partizioni presentano la stessa struttura a 36 colonne, comprendente feature numeriche, feature categoriche e due variabili target: `label` e `attack_cat`.

Per evitare forme di *data leakage*, tutte le analisi esplorative e le decisioni di preprocessing sono state definite esclusivamente sulla partizione di training. Il test set ufficiale è stato mantenuto separato e non utilizzato nella fase di esplorazione o selezione delle trasformazioni, preservandone il ruolo di valutazione finale indipendente, fatta eccezione per le operazioni standard di sanificazione strutturale (come la rimozione dei duplicati), necessarie per garantire l'affidabilità delle metriche.

La prima criticità rilevante emersa dall'EDA riguarda la presenza di righe duplicate esatte. Nel training set sono state individuate 78.519 righe duplicate successive alla prima occorrenza, pari al 44,78% dei record iniziali. Considerando anche la prima occorrenza di ciascun gruppo duplicato, 87.593 record, pari al 49,96% del training set, appartengono a gruppi di righe duplicate. Dopo la rimozione delle sole occorrenze duplicate, mantenendo la prima istanza, il training set risulta composto da 96.822 record.

L'analisi delle variabili target è stata condotta sul training set ripulito dai duplicati. Per il target binario `label`, la distribuzione risulta sostanzialmente bilanciata: 48.894 record normali, pari al 50,5%, e 47.928 record d'attacco, pari al 49,5%. La formulazione multiclasse basata su `attack_cat` mostra invece uno sbilanciamento marcato: la classe *Normal* rappresenta il 50,5% dei

record, seguita da *Exploits* con il 20,0%, *Fuzzers* con il 14,54% e *Reconnaissance* con il 6,2%. Le classi meno rappresentate includono *Shellcode*, pari allo 0,99%, e *Worms*, pari allo 0,13%. Questa evidenza ha motivato la scelta di considerare la classificazione binaria come task principale e la classificazione multiclasse come possibile estensione successiva.

Dal punto di vista della qualità dei dati, non sono stati rilevati valori mancanti espliciti. Tuttavia, la variabile categorica `service` contiene il valore “-”, interpretato non come missing value casuale, ma come indicatore semantico di servizio non identificato o non standard. Tale valore è stato quindi trattato esplicitamente nella fase di preprocessing.

Le statistiche descrittive hanno inoltre evidenziato distribuzioni fortemente asimmetriche in numerose feature numeriche. Sono state considerate altamente asimmetriche le variabili con skewness maggiore di 3. Tra queste rientrano `sbytes`, `dbytes`, `spkts`, `dpkts`, `sload`, `dload`, `dur`, `rate`, `sinpkt`, `dinpkt`, `sjit`, `djit`, `tcprrt`, `synack`, `ackdat`, `response_body_len`, `ct_ftp_cmd`, `ct_src_dport_ltm`, `ct_dst_sport_ltm`, `ct_flw_http_mthd` e `is_sm_ips_ports`. La presenza di code lunghe è coerente con il dominio del traffico di rete, nel quale pochi flussi possono assumere valori molto elevati in termini di durata, byte, pacchetti o rate.

Anche l’analisi degli outlier tramite criterio IQR ha evidenziato valori estremi in numerose feature. Le percentuali più elevate sono state osservate per variabili come `ct_src_dport_ltm` (22,34%), `dwin` (21,96%), `swin` (21,72%), `dbytes` (20,73%), `dmean` (19,88%), `ct_flw_http_mthd` (17,77%) e `sload` (16,06%).

Supervised Learning

Identificazione del Task e Gestione delle Variabili Target

Il problema viene formalizzato come un task di classificazione supervisionata. La formulazione principale adottata nel progetto è la classificazione binaria, nella quale l’obiettivo è distinguere tra traffico normale e traffico d’attacco tramite la variabile `label`. In tale codifica, il valore 0 rappresenta il traffico normale, mentre il valore 1 rappresenta il traffico malevolo.

Una seconda formulazione possibile è la classificazione multiclasse, basata sulla variabile `attack_cat`, che identifica la famiglia specifica dell’attacco. Questa formulazione è più complessa a causa del maggiore sbilanciamento tra le classi e della presenza di categorie rare. Per questo

motivo, nel presente stadio del lavoro essa viene mantenuta come estensione successiva, mentre la pipeline viene progettata principalmente in funzione del task binario.

Le caratteristiche delle variabili target sono riassunte nella Tabella 1.

Tabella 1: Variabili target considerate nel dataset UNSW-NB15. La tabella riassume il significato delle variabili obiettivo e il loro ruolo nella formulazione dei task predittivi.

Target	Significato	Tipo Dato	Task	Utilizzo nel progetto
label	Indicatore binario di anomalia (0: normale, 1: attacco).	Intero	Binario	Target principale della modellazione.
attack_cat	Categoria specifica dell'attacco informatico.	Categorico	Multiclasse	Preservata nel dataset pre-processato; esclusa dalle feature del task binario.

Ingegneria delle Feature e Metodologia di Preprocessing

La pipeline di preprocessing è stata progettata a partire dalle evidenze emerse durante l'EDA. Il primo passo consiste nella rimozione delle righe esattamente duplicate dal training set, mantenendo la prima occorrenza. Questa scelta mira a ridurre la ridondanza del dataset e a evitare che pattern ripetuti influenzino eccessivamente l'apprendimento dei modelli.

Successivamente, sono state rimosse alcune feature ridondanti individuate in fase di analisi, in particolare `trans_depth` e `is_ftp_login`. La prima è risultata fortemente associata a `ct_flw_http_mthd`, mentre la seconda è ridondante rispetto a `ct_ftp_cmd`. Tali rimozioni riducono la multicollinearità e semplificano lo spazio delle feature senza eliminare informazione sostanziale.

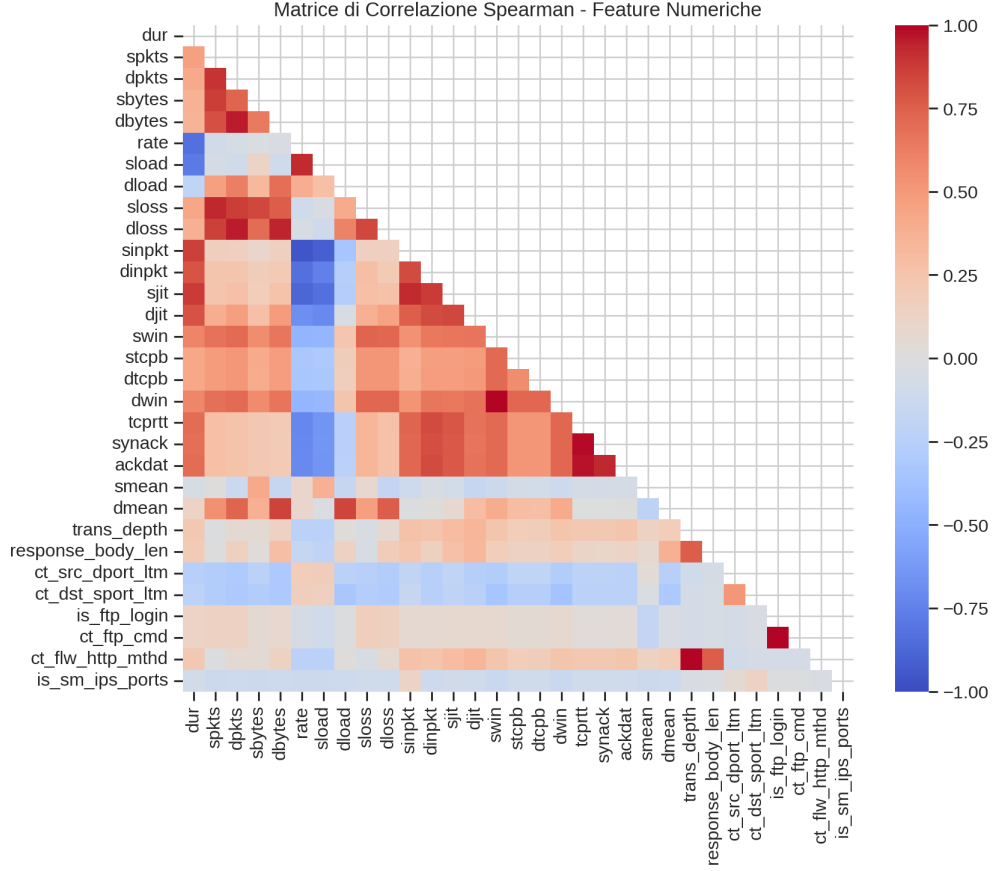


Figura 1: Matrice di correlazione di Spearman.

Per le feature categoriche, la colonna `service` contiene il valore “-”, che è stato ricodificato come “Unknown”. Le variabili `service` e `state` sono state successivamente trasformate tramite *One-Hot Encoding*. La variabile `proto`, caratterizzata da un numero elevato di categorie, è stata invece trattata con un passaggio preliminare di raggruppamento: i protocolli rari, identificati esclusivamente sulla base della frequenza nel training set, sono stati ricondotti alla categoria “Other” prima dell’applicazione del *One-Hot Encoding*. Questa scelta consente di ridurre la dimensionalità senza utilizzare informazione sul target.

Gli outlier rilevati non sono stati rimossi automaticamente, poiché nel dominio NIDS possono rappresentare segnali informativi associati ad anomalie o comportamenti malevoli. Le feature numeriche continue o di conteggio con skewness maggiore di 3 sono state trattate tramite trasformazioni logaritmiche:

$$X_{trasformato} = \ln(1 + X) \quad (1)$$

La trasformazione $\log_{1p}$ è adatta a variabili non negative e consente di comprimere code molto lunghe preservando i valori nulli. Nel dataset preprocessato, tale trasformazione è stata applicata alle feature numeriche altamente asimmetriche rimaste dopo la rimozione delle feature ridondanti.

Infine, il dataset preprocessato mantiene sia `label` sia `attack_cat`. Tuttavia, nella successiva fase di modellazione binaria, `attack_cat` è stata esclusa dalle feature predittive per evitare leakage, poiché contiene informazione diretta sulla natura dell'attacco. Al termine della pipeline, il training set preprocessato risulta composto da 96.822 record e 60 colonne complessive. Escludendo le due variabili target, `label` e `attack_cat`, lo spazio predittivo è costituito da 58 feature.

L'applicazione della pipeline di preprocessing al test set ufficiale è stata effettuata seguendo le decisioni definite sul training set, in modo da allineare le feature del test set a quelle del training preprocessato. L'unica modifica apportata al test set originale ha riguardato la rimozione delle osservazioni identiche presenti anche nel training set. Il confronto tra le due partizioni ha evidenziato 1.571 osservazioni uniche condivise, le cui occorrenze sono state eliminate esclusivamente dal test set prima dell'applicazione della pipeline di preprocessing, al fine di eliminare completamente qualsiasi forma di *data leakage* dovuta alla presenza di esempi identici nelle due partizioni. Complessivamente, tale operazione ha comportato la rimozione di 18.596 record, corrispondenti al 22,59% degli 82.332 record del test set ufficiale, riducendone la dimensione finale a 63.736 osservazioni.

Questa scelta comporta un lieve scostamento rispetto al benchmark ufficiale di UNSW-NB15, ma è stata ritenuta metodologicamente preferibile, poiché garantisce che la valutazione finale venga eseguita esclusivamente su osservazioni mai viste durante l'addestramento.

Standardizzazione delle Feature Numeriche

La standardizzazione delle feature è stata differenziata in base alla natura matematica dell'algoritmo impiegato. Per i modelli basati su iperpiani separatori e ottimizzazione convessa, ovvero la Regressione Logistica e la Support Vector Machine Lineare (Linear SVM), è stata applicata una standardizzazione robusta attraverso il RobustScaler. Questa trasformazione è definita matematicamente dalla seguente equazione:

$$x' = \frac{x - \text{mediana}}{\text{IQR}} \quad (2)$$

dove la variabile originale x viene centrata sottraendo la mediana della distribuzione e successivamente scalata dividendo per il Range Interquartile (IQR), calcolato come la differenza tra il terzo quartile (Q3, settantacinquesimo percentile) e il primo quartile (Q1, venticinquesimo percentile). L'utilizzo della mediana e del range interquartile conferisce a questa tecnica una marcata robustezza contro i valori anomali (outlier). In ambito di sicurezza informatica e nei Network Intrusion Detection Systems (NIDS), il traffico di rete presenta distribuzioni intrinsecamente asimmetriche ed eventi estremi fisiologici associati alle intrusioni. Il RobustScaler garantisce che tali outlier non distorcano il calcolo della media e della varianza, preservando l'integrità del segnale per i modelli lineari. Al contrario, gli algoritmi basati su ensemble di alberi decisionali, specificamente Random Forest, Extra Trees e XGBoost, non vengono sottoposti ad alcuna operazione di standardizzazione o normalizzazione. La struttura matematica degli alberi di decisione partiziona lo spazio delle feature tramite soglie di taglio (split) ortogonali basate sul guadagno di informazione o sull'impurità di Gini. Tale meccanismo risulta invariante alle trasformazioni monotone delle variabili di input, rendendo la standardizzazione superflua e mantenendo inalterata la scala originale dei dati per questi specifici classificatori.

K-Fold Cross-Validation e Selezione degli Iperparametri

L'individuazione della configurazione ottima degli iperparametri viene eseguita mediante una ricerca esaustiva nello spazio dei parametri (Grid Search), validata attraverso una rigorosa Stratified K-Fold Cross-Validation. Il dataset di addestramento viene diviso in $K = 5$ partizioni (fold). Questa suddivisione viene effettuata attivando un mescolamento preliminare dei campioni (shuffle) ancorato a un generatore di numeri pseudo-casuali a stato fisso, assicurando la totale riproducibilità dell'esperimento. La natura stratificata della partizione assicura che ogni fold riproduca esattamente la medesima distribuzione delle classi presente nel dataset originale, prevenendo distorsioni indotte da campionamenti sbilanciati. Il processo di validazione segue un flusso di lavoro strettamente iterativo e indipendente. Per ciascuna combinazione di iperparametri definita nella griglia, il modello viene addestrato ex novo per cinque volte consecutive. In ognuna di queste cinque iterazioni, quattro partizioni vengono aggregate per l'addestramento, mentre la rimanente viene impiegata per la validazione. Il punteggio associato alla specifica configurazione corrisponde alla media aritmetica delle prestazioni registrate nei cinque fold indipendenti. Di conseguenza, il numero totale

di addestramenti effettuati per un singolo classificatore è calcolato moltiplicando il numero delle combinazioni parametriche per il numero di fold ($K = 5$).

La Tabella 2 riassume la cardinalità dello spazio di ricerca e il conseguente carico computazionale per ciascun algoritmo analizzato. Ad esempio, la Regressione Logistica, valutata su 12 configurazioni derivate dalla combinazione dei parametri di regolarizzazione e bilanciamento delle classi, richiede $12 \times 5 = 60$ addestramenti indipendenti. Analogamente, la Linear SVM (6 combinazioni) genera 30 addestramenti, mentre Random Forest ed Extra Trees, con 16 combinazioni ciascuno, richiedono 80 iterazioni per modello. Infine, XGBoost esplora uno spazio di 32 combinazioni, rendendo necessari 160 addestramenti totali.

Tabella 2: Riepilogo delle configurazioni esplorate e del numero totale di addestramenti eseguiti durante la fase di Stratified K-Fold Cross-Validation ($K = 5$).

Modello	Numero combinazioni	Fold	Numero totale di training
Logistic Regression	12	5	60
Linear SVM	6	5	30
Random Forest	16	5	80
Extra Trees	16	5	80
XGBoost	32	5	160

Il criterio di selezione per identificare la migliore combinazione all'interno della griglia si basa esclusivamente sulla massimizzazione della media del punteggio F1 (F1-score) calcolata sui cinque fold. Al termine dell'intera procedura di ricerca esaustiva, la configurazione che ha registrato l'F1-score medio più elevato viene selezionata per un riaddestramento finale (refit). In questa fase conclusiva, l'algoritmo, parametrizzato con la configurazione ottima, viene addestrato una singola volta utilizzando l'interezza dei campioni disponibili nel training set, risultando così pronto per la valutazione sul set di test.

Metriche di Valutazione

Le prestazioni dei modelli vengono misurate utilizzando un insieme diversificato di metriche, formulate per quantificare in maniera esaustiva la capacità di rilevamento in contesti NIDS.

La metrica dell'Accuratezza descrive la frazione di predizioni corrette sull'intero dataset:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \quad (3)$$

dove TP rappresenta i Veri Positivi, TN i Veri Negativi, FP i Falsi Positivi e FN i Falsi Negativi.

Tuttavia, nei contesti di Network Intrusion Detection l'Accuratezza, pur fornendo una misura complessiva delle prestazioni, non risulta sufficiente a descrivere il comportamento del classificatore, poiché assume particolare rilevanza il compromesso tra falsi negativi e falsi positivi.

Per ovviare a questo problema, la valutazione principale in contesti di Network Intrusion Detection si concentra sulla Recall (Sensibilità):

$$\text{Recall} = \frac{TP}{TP + FN} \quad (4)$$

Nei sistemi IDS, la Recall assume un'importanza fondamentale in quanto quantifica la capacità del sistema di individuare tutto il traffico malevolo effettivo, minimizzando i falsi negativi che si tradurrebbero in intrusioni non identificate all'interno dell'infrastruttura di rete.

Simultaneamente, viene calcolata la Precision:

$$\text{Precision} = \frac{TP}{TP + FP} \quad (5)$$

La Precision risulta cruciale per limitare la quantità di falsi positivi. Un volume eccessivo di allarmi errati genererebbe infatti un fenomeno di Alert Fatigue, degradando l'efficienza operativa degli analisti di sicurezza costretti a indagare su segnalazioni non corrispondenti a minacce reali.

Per bilanciare Precision e Recall, il modello viene ottimizzato e valutato attraverso l'F1-score, definito come la media armonica tra Precision e Recall:

$$\text{F1-score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \quad (6)$$

L'F1-score penalizza i modelli che presentano forti sbilanciamenti tra le due metriche, motivo per il quale è stato selezionato come funzione obiettivo primario per la selezione degli iperparametri durante la Grid Search.

Le capacità probabilistiche del classificatore sono analizzate tramite due aree sotto la curva. La prima è l'Area Sotto la Curva ROC (ROC-AUC), che confronta il tasso di veri positivi (TPR) con il tasso di falsi positivi (FPR). La seconda è l'Area Sotto la Curva Precision-Recall (PR-AUC), che

valuta il compromesso diretto tra Precision e Recall al variare della soglia decisionale. Entrambe vengono riportate per completezza analitica, tuttavia nei sistemi di Network Intrusion Detection la PR-AUC è generalmente considerata più rappresentativa poiché sintetizza il comportamento del classificatore rispetto a Precision e Recall, metriche direttamente associate alla riduzione dei falsi negativi e dei falsi positivi.

Infine, l'analisi qualitativa e granulare degli errori commessi dal modello viene condotta tramite l'ispezione della Confusion Matrix. Questa rappresentazione tabellare degli indicatori di base (TP, TN, FP, FN) permette di osservare visivamente le dinamiche di misclassificazione del classificatore, distinguendo esattamente quanti eventi benigni vengono scambiati per minacce e viceversa.

Apprendimento Non Supervisionato mediante K-Means

Al fine di esplorare le strutture latenti intrinseche al traffico di rete, la classificazione supervisionata è stata affiancata da un'analisi di apprendimento non supervisionato basata sull'algoritmo K-Means. A differenza dei paradigmi supervisionati, il clustering non supervisionato opera in totale assenza di etichette (label) durante la fase di addestramento, raggruppando i flussi di rete basandosi unicamente sulle loro affinità topologiche nello spazio delle feature. Nel dominio dei Network Intrusion Detection Systems (NIDS), questa tipologia di analisi risulta particolarmente preziosa in quanto permette di estrarre pattern di traffico nascosti e identificare cluster di anomalie che potrebbero non coincidere perfettamente con le firme degli attacchi noti. Pertanto, il clustering rappresenta un'analisi complementare che arricchisce la modellazione supervisionata, offrendo una prospettiva geometrica sulla distribuzione delle minacce.

Standardizzazione dei Dati

Essendo K-Means un algoritmo intrinsecamente basato sul calcolo delle distanze euclidee tra le osservazioni e i centroidi, risulta particolarmente sensibile alla scala delle feature. Per questo motivo è stata applicata una fase di standardizzazione preliminare, così da rendere confrontabili le diverse variabili e impedire che quelle caratterizzate da valori numerici più elevati influenzassero in modo sproporzionato il processo di clustering.

Diversamente dalla fase di classificazione supervisionata, in questo esperimento non è stato

utilizzato *RobustScaler*. L'analisi esplorativa ha infatti evidenziato che le feature *swin* e *dwin*, caratterizzate da un intervallo interquartile nullo e da valori prevalentemente pari a 0 e 255, producevano una forte predominanza geometrica dopo tale trasformazione. Poiché K-Means basa il processo di clustering esclusivamente sulle distanze tra le osservazioni, tale effetto alterava significativamente la struttura dello spazio delle feature. È stato pertanto adottato *StandardScaler*, che utilizza la deviazione standard come fattore di scala anziché l'intervallo interquartile, evitando questa distorsione e garantendo un contributo più equilibrato delle diverse variabili nel calcolo delle distanze.

La trasformazione, applicata a ogni singola osservazione x , è definita matematicamente come:

$$x' = \frac{x - \mu}{\sigma} \quad (7)$$

dove μ rappresenta la media campionaria della specifica feature sul training set e σ la relativa deviazione standard. Questa operazione di *scaling* garantisce che ogni variabile possieda media nulla e varianza unitaria, normalizzando il peso spaziale di ciascuna dimensione e assicurando un processo di raggruppamento geometricamente equilibrato.

Configurazione del Modello e Workflow Sperimentale

La configurazione del modello K-Means ha previsto l'impostazione di un generatore di numeri pseudo-casuali a stato fisso per garantire la riproducibilità, unita a un numero di inizializzazioni indipendenti dei centroidi pari a dieci ($n_{init} = 10$), al fine di scongiurare la convergenza dell'algoritmo verso minimi locali sub-ottimali. Per esplorare la granularità topologica del dataset, il numero di cluster, indicato con k , è stato sistematicamente variato analizzando tutti i valori discreti compresi tra $k = 2$ e $k = 10$. L'indagine di differenti valori di k permette di determinare sperimentalmente quale configurazione rifletta con maggiore fedeltà la separazione naturale tra traffico legittimo e molteplici famiglie di attacco.

Il workflow sperimentale è stato rigorosamente strutturato per separare la fase di apprendimento da quella di validazione. Per ogni valore di k , il modello K-Means è stato addestrato esclusivamente sul training set, senza alcun accesso alle etichette di verità (*ground truth*). Successivamente, i cluster appresi sono stati assegnati sia ai campioni del training set stesso sia ai campioni del test set, propagando le logiche spaziali del modello sui dati non visti durante l'addestramento.

Scenari Sperimentali

Per fornire un'analisi robusta e coerente con i task supervisionati, l'apprendimento non supervisionato è stato declinato in due distinti scenari operativi. In ogni scenario, la variabile target e le eventuali feature strettamente correlate ad essa sono state categoricamente omesse dallo spazio di input. Questa esclusione preventiva è imperativa per evitare qualsiasi forma di *Data Leakage*, impedendo all'algoritmo di dedurre banalmente la separazione basandosi sulla risposta desiderata.

Nel primo scenario, volto a simulare un contesto di classificazione binaria, il target di riferimento per la validazione è stato l'indicatore di anomalia globale (`label`), con la conseguente esclusione della variabile `attack_cat`. Nel secondo scenario, mirato alla risoluzione multiclasse, il target di validazione è stato la categoria specifica dell'attacco (`attack_cat`), comportando la completa esclusione della variabile binaria `label`. Poiché ogni scenario è stato iterato per tutte le nove configurazioni di k (da 2 a 10), sono stati eseguiti complessivamente 18 esperimenti indipendenti. Tale struttura sperimentale è riassunta nella Tabella 3.

Tabella 3: Riepilogo degli scenari sperimentali indipendenti configurati per la valutazione dell'algoritmo K-Means.

Scenario	Target	Variabile esclusa	Valori di k	Numero esperimenti
Scenario 1	label	attack_cat	2–10	9
Scenario 2	attack_cat	label	2–10	9
Totale	—	—	—	18

Associazione Cluster - Classe

Per consentire una comparazione quantitativa diretta tra le partizioni individuate dall'algoritmo non supervisionato e le etichette reali, è stata implementata una procedura di associazione basata sul voto di maggioranza (*majority voting*). Al termine del processo di addestramento e della conseguente partizione del training set, a ciascun cluster C_i è stata associata l'etichetta della classe predominante calcolata esclusivamente sui campioni appartenenti a tale cluster nel training set.

La mappatura così ottenuta, che traduce l'identificativo numerico del cluster nella classe reale ad esso più affine, è stata successivamente traslata e applicata ai raggruppamenti assegnati al

test set. Questo procedimento assicura che il calcolo delle metriche esterne avvenga in modo metodologicamente corretto: il processo di clustering rimane del tutto cieco alle etichette, le quali intervengono unicamente in fase di valutazione a posteriori per attribuire un significato semantico alle regioni dello spazio vettoriale identificate dal modello.

Metriche Interne di Clustering

La qualità strutturale dei raggruppamenti è stata misurata impiegando tre metriche interne, prive di qualsiasi dipendenza dalle etichette reali:

Il primo indicatore adottato è il *Silhouette Score*. Esso quantifica contemporaneamente la coesione intra-cluster e la separazione inter-cluster. Per un singolo campione i , è definito come:

$$S(i) = \frac{b(i) - a(i)}{\max(a(i), b(i))} \quad (8)$$

dove $a(i)$ rappresenta la distanza media tra i e gli altri punti dello stesso cluster, mentre $b(i)$ indica la distanza media minima tra i e i punti del cluster più vicino. Il valore complessivo varia tra -1 e $+1$; valori prossimi a $+1$ denotano un clustering denso e ben separato, mentre valori negativi suggeriscono un'errata assegnazione geometrica.

Il secondo indicatore è l'Indice di *Davies-Bouldin* (DB). Tale metrica si basa sul rapporto tra la dispersione interna dei cluster e la distanza euclidea tra i loro centroidi:

$$DB = \frac{1}{k} \sum_{i=1}^k \max_{j \neq i} \left(\frac{S_i + S_j}{d(c_i, c_j)} \right) \quad (9)$$

dove S_i quantifica la dispersione del cluster i e $d(c_i, c_j)$ la distanza tra i centroidi. Da un punto di vista geometrico, un basso valore del Davies-Bouldin Index è desiderabile in quanto indica che i cluster presentano un'alta compattezza interna unita a una chiara separazione spaziale.

Infine, è stato calcolato l'Indice di *Calinski-Harabasz* (CH), o criterio del rapporto della varianza, espresso come:

$$CH = \frac{\text{Tr}(B_k)}{\text{Tr}(W_k)} \times \frac{N - k}{k - 1} \quad (10)$$

dove $\text{Tr}(B_k)$ rappresenta la traccia della matrice di dispersione inter-cluster, $\text{Tr}(W_k)$ la traccia della dispersione intra-cluster, N il numero totale di campioni e k il numero di cluster. Valori elevati del CH Index riflettono una marcata separazione e raggruppamenti ben definiti.

Metriche Esterne di Classificazione

La validazione incrociata con le etichette reali (dopo l'associazione cluster-classe) ha sfruttato le medesime metriche esterne adottate per i modelli supervisionati. Nel primo scenario (binario), le prestazioni sono state quantificate calcolando Accuracy, Precision, Recall e F1-score. Nel secondo scenario (multiclasse), a fronte dello sbilanciamento nella frequenza delle famiglie di attacco, l'Accuracy globale è stata affiancata dalle versioni Macro e Weighted di Precision, Recall e F1-score. Le metriche Macro assegnano egual peso a ogni classe, premiando la capacità del clustering di isolare minacce rare, mentre le metriche Weighted calibrano il punteggio in proporzione alla cardinalità di ciascuna classe.

Risultati sperimentali

In questo capitolo vengono presentati i risultati ottenuti dalla valutazione dei modelli di machine learning sviluppati per il problema di *Network Intrusion Detection* sul dataset UNSW-NB15, esplorando sia approcci supervisionati che un metodo non supervisionato. Tutti gli algoritmi sono stati addestrati utilizzando la medesima pipeline di preprocessing; in particolare, i modelli supervisionati sono stati ottimizzati mediante Grid Search con validazione incrociata, così da garantire un confronto equo. Come descritto nel capitolo metodologico, tutte le operazioni di preprocessing, selezione degli iperparametri e addestramento (inclusa la definizione dello spazio dei cluster) sono state definite esclusivamente sul training set, mantenendo completamente indipendente il test set finale ed evitando qualsiasi forma di *data leakage*.

L'analisi sperimentale prende in esame cinque algoritmi supervisionati appartenenti a differenti famiglie — Logistic Regression, Linear Support Vector Machine (Linear SVM), Extra Trees, Random Forest e XGBoost — ai quali si affianca un'ulteriore analisi basata sull'algoritmo non supervisionato K-Means. Le prestazioni dei modelli sono state valutate attraverso un insieme di metriche complementari, comprendenti Accuracy, Precision, Recall, F1-score, ROC-AUC e PR-AUC. L'utilizzo di più indicatori consente una valutazione più completa rispetto alla sola accuracy, particolarmente importante nel contesto dell'*Intrusion Detection*, dove l'obiettivo non è soltanto massimizzare il numero di classificazioni corrette, ma anche limitare contemporaneamente i falsi positivi e i falsi negativi.

Confronto tra i modelli supervisionati

La Tabella 4 riassume le prestazioni ottenute dai modelli supervisionati sul test set indipendente.

Tabella 4: Prestazioni dei modelli supervisionati sul test set indipendente.

Modello	Accuracy	Precision	Recall	F1-score	ROC-AUC	PR-AUC
XGBoost	0.851	0.779	0.967	0.863	0.975	0.977
Random Forest	0.839	0.761	0.971	0.853	0.974	0.976
Extra Trees	0.819	0.735	0.981	0.840	0.970	0.972
Linear SVM	0.772	0.691	0.955	0.802	0.878	0.854
Logistic Regression	0.769	0.692	0.940	0.797	0.888	0.882

L'analisi dei risultati evidenzia una netta superiorità dei metodi basati su ensemble di alberi rispetto ai classificatori lineari. In particolare, **XGBoost** rappresenta il modello con le migliori prestazioni complessive, raggiungendo un'Accuracy pari all'85.15%, un F1-score di circa 0.86 e valori di ROC-AUC e PR-AUC superiori a 0.97. Tali risultati indicano un'elevata capacità discriminativa tra traffico benigno e traffico malevolo, mantenendo un buon equilibrio tra Precision e Recall.

Il secondo miglior classificatore risulta essere **Random Forest**, che ottiene prestazioni molto vicine a quelle di XGBoost. La differenza contenuta tra i due algoritmi suggerisce come entrambi siano particolarmente adatti alla classificazione di dati tabellari complessi, caratterizzati da relazioni non lineari e interazioni tra variabili. Tuttavia, il meccanismo di boosting adottato da XGBoost, che corregge iterativamente gli errori commessi dagli alberi precedenti, consente di ottenere un miglioramento misurabile delle prestazioni finali.

Anche **Extra Trees** mostra risultati competitivi, raggiungendo un F1-score superiore a 0.84. È interessante osservare come questo modello presenti il valore di Recall più elevato tra tutti i classificatori analizzati, evidenziando una notevole capacità di individuare il traffico malevolo. Tale comportamento è tuttavia accompagnato da una lieve riduzione della Precision, indice di un numero maggiore di falsi positivi rispetto agli altri ensemble.

Diversamente, i classificatori lineari (**Logistic Regression** e **Linear SVM**) mostrano prestazioni

sensibilmente inferiori. Entrambi raggiungono un'Accuracy di circa il 77% e valori di F1-score prossimi a 0.80, mantenendo comunque valori di Recall relativamente elevati. Questo comportamento suggerisce che tali algoritmi riescano a identificare una buona parte degli attacchi, ma incontrino maggiori difficoltà nel modellare le relazioni altamente non lineari presenti nello spazio delle caratteristiche del dataset UNSW-NB15, producendo un numero significativamente superiore di classificazioni errate rispetto agli approcci basati su alberi decisionali.

I risultati ottenuti sono pienamente coerenti con quanto emerso dalla letteratura analizzata nel Capitolo 2. Diversi studi evidenziano infatti come gli algoritmi ensemble basati su alberi decisionali rappresentino lo stato dell'arte per la classificazione di dati tabellari nel contesto dell'Intrusion Detection. La loro capacità di modellare interazioni non lineari, gestire efficacemente variabili eterogenee e risultare intrinsecamente robusti alla presenza di outlier permette di ottenere prestazioni superiori rispetto ai modelli lineari, la cui capacità rappresentativa risulta limitata quando le frontiere decisionali assumono una struttura particolarmente complessa.

È inoltre opportuno osservare che le prestazioni ottenute durante la fase di cross-validation risultano sistematicamente superiori rispetto a quelle misurate sul test set indipendente. Tale differenza evidenzia un fisiologico calo della capacità di generalizzazione quando i modelli vengono valutati su dati completamente mai osservati durante le fasi di addestramento e selezione degli iperparametri. Nel caso specifico, il divario tra le metriche di validazione e quelle del test set rimane comunque contenuto e non suggerisce fenomeni di overfitting particolarmente pronunciati, risultando coerente con quanto generalmente atteso in un protocollo sperimentale basato su ottimizzazione mediante Grid Search e successiva valutazione su un insieme di dati indipendente. La Tabella 5 riporta inoltre i tempi di addestramento e di inferenza dei modelli considerati, consentendo di confrontarne non soltanto l'efficacia predittiva, ma anche l'efficienza computazionale.

Analisi delle matrici di confusione

Oltre alle metriche aggregate riportate nella Tabella 4, l'analisi delle matrici di confusione consente di comprendere più nel dettaglio il comportamento dei diversi classificatori, evidenziando la distribuzione delle predizioni corrette ed errate tra le due classi considerate. In particolare, la matrice di confusione permette di quantificare il numero di *True Positive* (TP), *True Negative* (TN),

Tabella 5: Tempi di addestramento e inferenza dei modelli supervisionati.

Modello	Training (s)	Inferenza (s)
XGBoost	83.54	0.138
Random Forest	579.05	0.764
Extra Trees	225.04	0.554
Linear SVM	73.12	0.066
Logistic Regression	1460.98	0.108

False Positive (FP) e *False Negative* (FN), fornendo una rappresentazione immediata dei principali errori di classificazione.

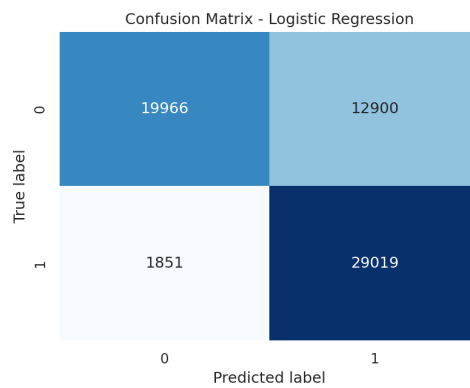


Figura 2: Matrice di confusione del modello Logistic Regression.

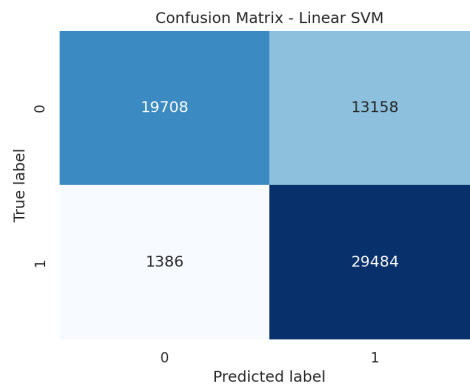


Figura 3: Matrice di confusione del modello Linear SVM.

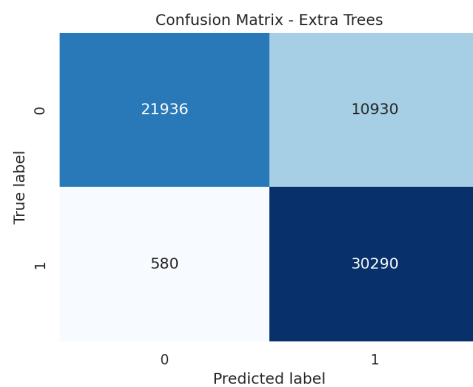


Figura 4: Matrice di confusione del modello Extra Trees.

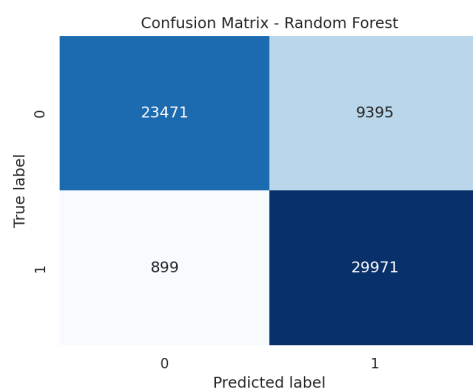


Figura 5: Matrice di confusione del modello Random Forest.

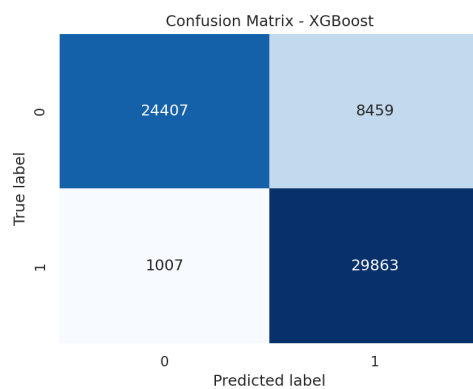


Figura 6: Matrice di confusione del modello XGBoost.

L'osservazione delle matrici di confusione conferma quanto emerso dall'analisi quantitativa delle metriche. Tutti i modelli mostrano un'elevata capacità di identificare il traffico malevolo, come evidenziato dall'elevato numero di veri positivi e dai valori di Recall superiori al 94% per tutti i classificatori considerati. Questo comportamento risulta particolarmente importante nel contesto dell'Intrusion Detection, dove la mancata identificazione di un attacco (*False Negative*) rappresenta generalmente l'errore più critico.

Le differenze tra i modelli emergono principalmente nella gestione del traffico benigno. I classificatori lineari, pur mantenendo un'elevata sensibilità verso gli attacchi, producono un numero significativamente maggiore di falsi positivi rispetto agli algoritmi basati su ensemble di alberi. Tale comportamento è coerente con i valori di Precision osservati nella Tabella 4 e suggerisce una minore capacità dei modelli lineari di rappresentare la complessità delle relazioni presenti nello spazio delle caratteristiche del dataset.

Gli algoritmi ensemble mostrano invece matrici di confusione maggiormente bilanciate. In particolare, XGBoost e Random Forest riducono sensibilmente il numero di classificazioni errate sia sulla classe normale sia sulla classe di attacco, mantenendo contemporaneamente un'elevata capacità di rilevamento delle intrusioni. Extra Trees presenta il valore di Recall più elevato, intercettando un numero leggermente superiore di attacchi, ma a fronte di un incremento dei falsi positivi, comportamento che riflette il compromesso tra Recall e Precision già evidenziato dalle metriche aggregate.

Nel complesso, l'analisi delle matrici di confusione conferma che i modelli basati su ensemble riescono a mantenere un migliore equilibrio tra sensibilità e specificità, riducendo simultaneamente sia i falsi positivi sia i falsi negativi. Tale risultato risulta particolarmente rilevante in un contesto operativo, nel quale un numero eccessivo di falsi positivi aumenta il carico di lavoro degli analisti di sicurezza, mentre i falsi negativi possono comportare il mancato rilevamento di attacchi reali.

Analisi delle curve ROC e Precision-Recall

Per completare la valutazione dei modelli supervisionati sono state analizzate congiuntamente le curve *Receiver Operating Characteristic* (ROC) e *Precision-Recall* (PR), riportate nelle Figure 7–11. A differenza delle metriche puntuali, queste rappresentazioni consentono di valutare

il comportamento dei classificatori al variare della soglia decisionale, fornendo una visione più completa della loro capacità discriminativa.

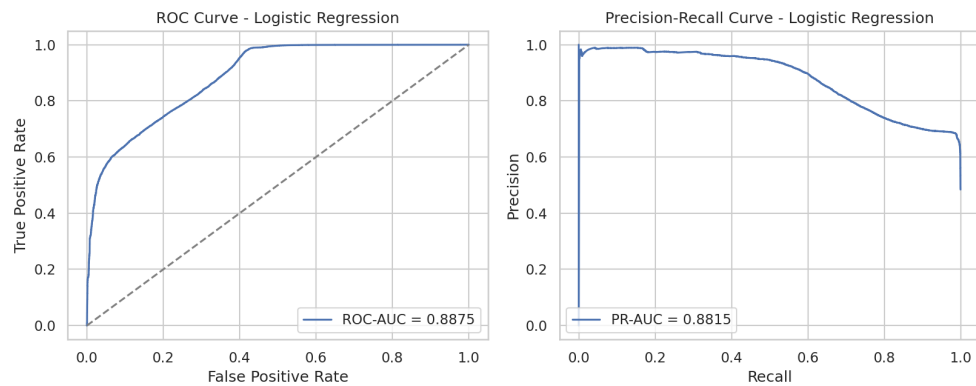


Figura 7: Curve ROC e Precision-Recall del modello Logistic Regression.

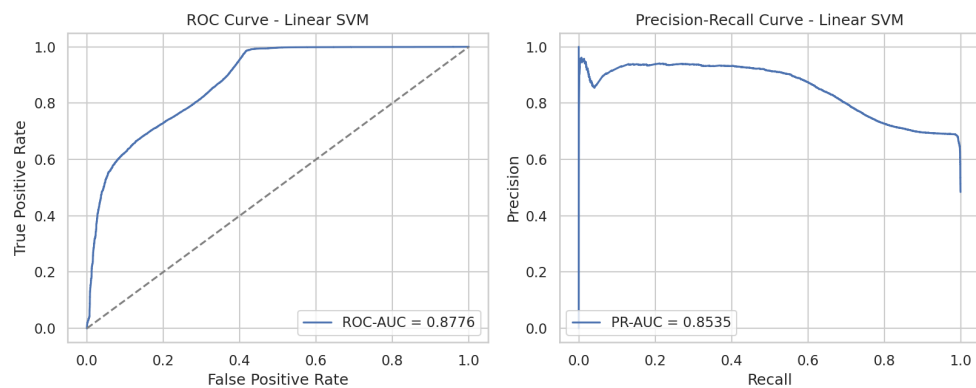


Figura 8: Curve ROC e Precision-Recall del modello Linear SVM.

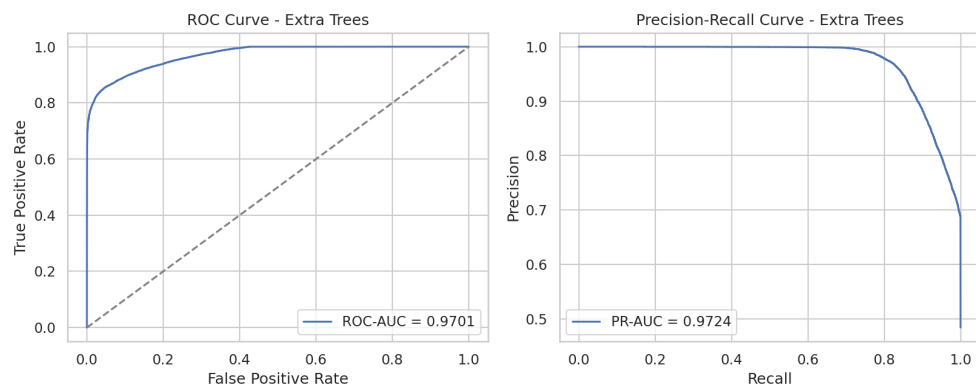


Figura 9: Curve ROC e Precision-Recall del modello Extra Trees.

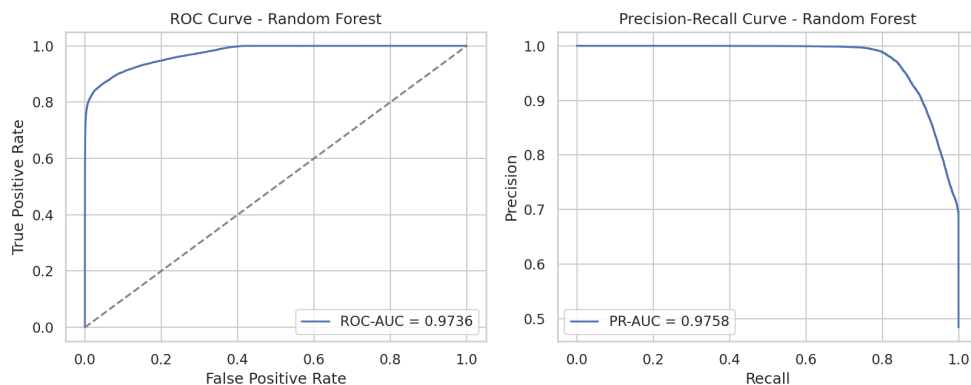


Figura 10: Curve ROC e Precision-Recall del modello Random Forest.

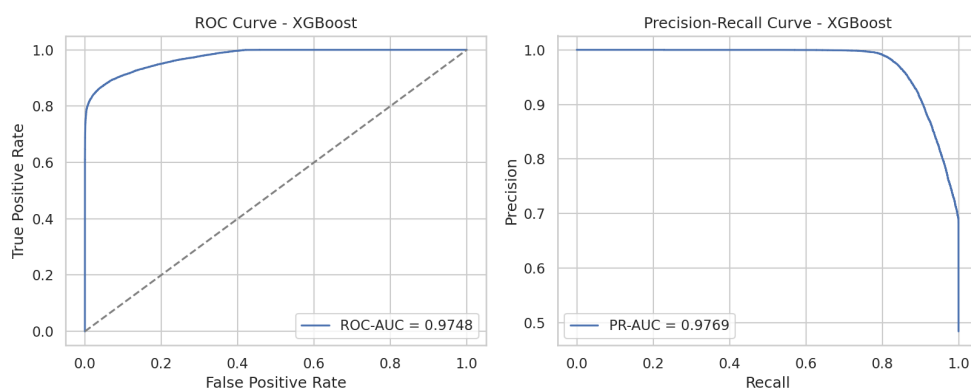


Figura 11: Curve ROC e Precision-Recall del modello XGBoost.

L'analisi delle curve conferma quanto già emerso dalle metriche quantitative riportate nella Tabella 4. I modelli basati su ensemble presentano curve ROC prossime all'angolo superiore sinistro del grafico e curve Precision-Recall caratterizzate da valori di Precision elevati lungo gran parte dell'intervallo di Recall, evidenziando un'eccellente capacità di distinguere il traffico benigno da quello malevolo indipendentemente dalla soglia decisionale adottata.

In particolare, XGBoost mantiene le prestazioni migliori sia in termini di ROC-AUC sia di PR-AUC, seguito da Random Forest ed Extra Trees. La maggiore area sottesa alle curve testimonia una superiore capacità discriminativa e una migliore gestione del compromesso tra identificazione degli attacchi e contenimento dei falsi positivi. Extra Trees, coerentemente con quanto osservato nell'analisi delle matrici di confusione, privilegia leggermente la Recall rispetto alla Precision, intercettando un numero maggiore di attacchi a fronte di un incremento dei falsi allarmi.

Le curve associate ai classificatori lineari mostrano invece un decadimento più rapido della Precision all'aumentare della Recall e una minore vicinanza all'angolo ideale della curva ROC. Tale comportamento riflette la maggiore difficoltà di Logistic Regression e Linear SVM nel modellare le relazioni non lineari presenti nel dataset UNSW-NB15, confermando i risultati quantitativi precedentemente discussi.

È opportuno osservare come l'analisi congiunta delle curve ROC e Precision-Recall risulti particolarmente significativa nel contesto dell'Intrusion Detection. Come discusso nella review della letteratura, la ROC-AUC fornisce una misura globale della capacità discriminativa del classificatore, mentre la PR-AUC offre una valutazione maggiormente rappresentativa nei problemi caratterizzati da distribuzioni sbilanciate, poiché risulta meno influenzata dall'elevato numero di veri negativi. La coerenza tra entrambe le curve rafforza pertanto le conclusioni ottenute attraverso le metriche aggregate, confermando la superiorità degli algoritmi ensemble rispetto ai modelli lineari per la classificazione del traffico di rete.

Analisi dell'importanza delle feature

Per comprendere quali caratteristiche del traffico di rete abbiano maggiormente influenzato le decisioni dei modelli supervisionati, è stata analizzata l'importanza delle feature calcolata per ciascun classificatore. Per gli algoritmi basati su alberi (Extra Trees, Random Forest e XGBoost) è stata considerata l'importanza derivata dalla riduzione dell'impurità durante la costruzione degli alberi decisionali, mentre per Logistic Regression e Linear SVM è stato analizzato il valore assoluto dei coefficienti appresi dal modello. Poiché gli algoritmi tree-based hanno ottenuto le migliori prestazioni complessive e forniscono una misura di importanza maggiormente interpretabile nel contesto della classificazione del traffico di rete, nelle figure seguenti vengono riportati esclusivamente i grafici relativi a Extra Trees, Random Forest e XGBoost. I coefficienti dei modelli lineari sono stati comunque analizzati e utilizzati a fini interpretativi nella discussione dei risultati.

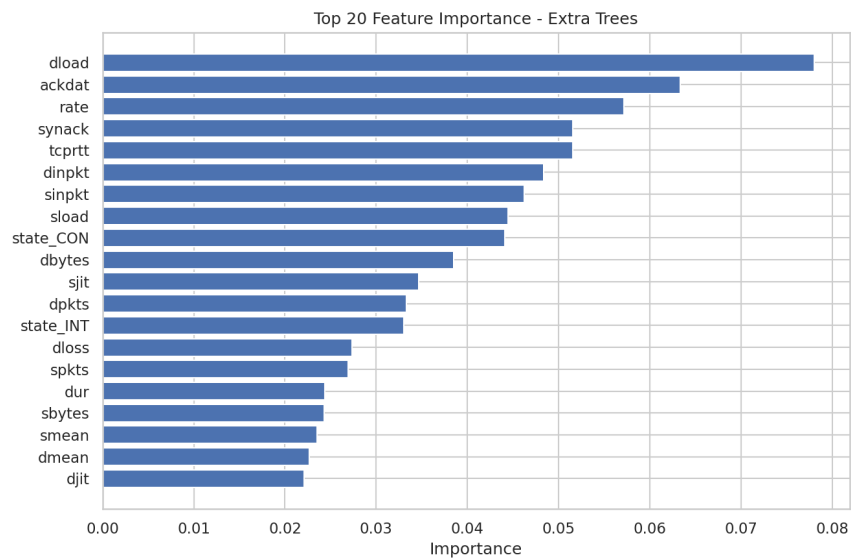


Figura 12: Importanza delle feature stimata dal modello Extra Trees.

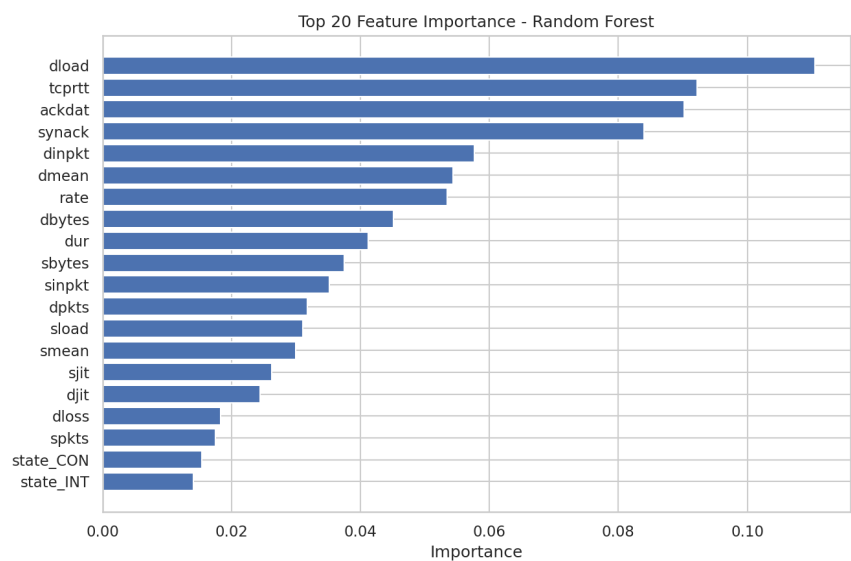


Figura 13: Importanza delle feature stimata dal modello Random Forest.

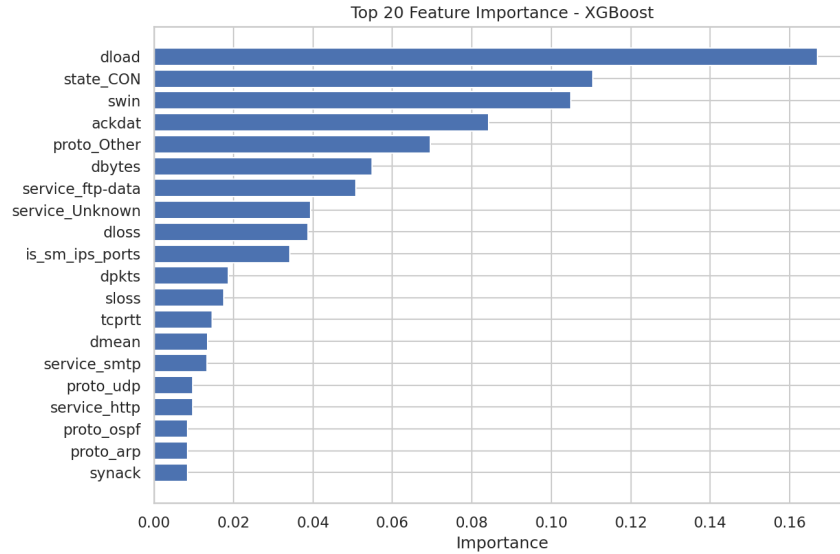


Figura 14: Importanza delle feature stimata dal modello XGBoost.

L'analisi evidenzia una notevole coerenza tra gli algoritmi appartenenti alla stessa famiglia. In particolare, i modelli ensemble identificano sistematicamente *dload* come una delle variabili maggiormente informative, risultando la feature più importante sia per Random Forest sia per Extra Trees e assumendo un peso ancora più marcato nel modello XGBoost. Tale risultato suggerisce che il volume di dati ricevuti rappresenti uno degli indicatori più efficaci per discriminare tra traffico benigno e attività malevole nel dataset UNSW-NB15.

Un secondo gruppo di variabili particolarmente rilevanti è costituito dalle caratteristiche associate al comportamento delle connessioni TCP, tra cui *tcprtt*, *synack* e *ackdat*. Queste feature compaiono sistematicamente tra le più importanti sia nei modelli lineari sia negli algoritmi ensemble, indicando come i tempi di risposta della connessione e le dinamiche di instaurazione della sessione TCP forniscano informazioni altamente discriminanti per il riconoscimento delle intrusioni.

Nel caso di XGBoost emerge inoltre una maggiore capacità di sfruttare variabili categoriche codificate, come *state_CON*, *proto_Other*, *service_ftp-data* e *service_Unknown*. Questo comportamento riflette la capacità degli algoritmi di boosting di modellare interazioni non lineari tra feature numeriche e categoriche, contribuendo probabilmente al miglioramento delle prestazioni osservato rispetto agli altri classificatori.

I modelli lineari mostrano invece una distribuzione dell'importanza maggiormente concentrata su un numero limitato di variabili. Logistic Regression e Linear SVM attribuiscono infatti il peso maggiore a *tcprtt*, seguito da *synack* e *ackdat*, mentre le restanti feature presentano coefficienti progressivamente inferiori. Tale comportamento è coerente con la natura lineare di questi algoritmi, che descrivono il contributo delle variabili attraverso una combinazione lineare e risultano meno efficaci nel rappresentare interazioni complesse tra le caratteristiche del traffico di rete.

Nel complesso, l'analisi dell'importanza delle feature conferma quanto emerso nella review della letteratura. Gli algoritmi basati su alberi non solo ottengono prestazioni predittive superiori, ma consentono anche di individuare con maggiore efficacia le variabili maggiormente informative per il problema di classificazione. La presenza ricorrente delle stesse feature tra i modelli migliori suggerisce inoltre che tali caratteristiche rappresentino effettivamente gli attributi più discriminanti del dataset, piuttosto che semplici artefatti del processo di addestramento.

Feature Selection

Al termine della fase di confronto tra i modelli supervisionati è stata condotta un'ulteriore analisi di *feature selection* applicata esclusivamente al classificatore che ha ottenuto le migliori prestazioni complessive nella fase di valutazione, ossia XGBoost. L'obiettivo di questa fase non era individuare il miglior modello, già selezionato nella precedente procedura di ottimizzazione, bensì valutare se fosse possibile ridurre il numero di variabili in ingresso mantenendo prestazioni comparabili e ottenendo un modello più compatto e potenzialmente più efficiente.

La selezione delle feature è stata effettuata sfruttando direttamente le informazioni di importanza apprese dal modello XGBoost già addestrato durante la fase di modellazione. In particolare, è stato utilizzato il metodo `SelectFromModel` della libreria `scikit-learn` con l'opzione `prefit=True`, che consente di riutilizzare il modello precedentemente addestrato senza eseguire un nuovo processo di apprendimento per il calcolo delle importanze. In questo modo, la selezione delle feature è stata interamente guidata dalle *feature importance* già stimate dal modello finale, evitando qualsiasi forma di riaddestramento nella fase di selezione.

Per analizzare l'influenza della soglia di selezione sul compromesso tra complessità del modello e prestazioni predittive sono state considerate diverse soglie di importanza (`mean`, `median`,

1.25*mean, 1.5*mean, 1.75*mean e 2*mean). Per ciascuna soglia sono state mantenute esclusivamente le variabili con un'importanza superiore alla soglia specificata, ottenendo così differenti sottoinsiemi di feature.

Successivamente, per ogni sottoinsieme selezionato è stato addestrato un nuovo classificatore XGBoost utilizzando esattamente gli stessi iperparametri individuati nella precedente fase di ottimizzazione tramite *Grid Search*. Tale scelta ha consentito di isolare l'effetto della riduzione delle feature, mantenendo inalterata la configurazione del modello e garantendo la confrontabilità tra gli esperimenti.

Ogni nuovo modello è stato quindi valutato sul medesimo test set utilizzato nella valutazione finale dei classificatori supervisionati. Sono state calcolate le stesse metriche impiegate nella fase di confronto dei modelli, ovvero Accuracy, Precision, Recall, F1-score, ROC-AUC e PR-AUC. Inoltre, è stato misurato il tempo medio di inferenza mediante venti esecuzioni consecutive della fase di predizione sull'intero test set, calcolandone successivamente il valore medio. Per garantire un confronto metodologicamente corretto, le stesse metriche e il tempo medio di inferenza sono stati preliminarmente ricalcolati anche per il modello XGBoost originale, utilizzando l'intero insieme delle feature, così da disporre di una baseline ottenuta nelle medesime condizioni sperimentali.

Tabella 6: Risultati della feature selection applicata al modello XGBoost.

Soglia	# Feature	Accuracy	F1-score	ROC-AUC	PR-AUC	Inferenza (s)
Baseline	58	0.851	0.863	0.975	0.977	0.136
Mean	12	0.814	0.836	0.959	0.958	0.112
Median	29	0.853	0.864	0.975	0.977	0.118
1.25×Mean	10	0.809	0.832	0.957	0.956	0.112
1.5×Mean	10	0.809	0.832	0.957	0.956	0.113
1.75×Mean	10	0.809	0.832	0.957	0.956	0.114
2×Mean	9	0.810	0.833	0.955	0.954	0.115

La Tabella 6 riporta le prestazioni ottenute dal modello XGBoost dopo l'applicazione della procedura di feature selection sopra descritta. I risultati sono confrontati con il modello originale addestrato utilizzando l'intero insieme delle 58 feature, impiegato come baseline sperimentale.

L'analisi evidenzia come la soglia corrispondente alla *median* rappresenti il miglior compromesso tra riduzione della dimensionalità e mantenimento delle prestazioni predittive. Con questa configurazione il numero di feature viene infatti ridotto del 50%, passando da 58 a 29 variabili, senza alcun degrado significativo delle metriche di classificazione. Al contrario, si osserva un lieve incremento dell'Accuracy (85.29% contro 85.15%) e dell'F1-score (0.864 contro 0.863), mentre ROC-AUC e PR-AUC rimangono sostanzialmente invariate, con differenze trascurabili rispetto alla baseline. Tali risultati indicano che una parte delle feature originarie fornisce informazioni ridondanti o scarsamente informative e che il classificatore fosse in grado di mantenere pressoché invariata la propria capacità discriminativa utilizzando un insieme significativamente più compatto di variabili.

Diversamente, l'impiego di soglie più restrittive determina una riduzione ancora maggiore del numero di feature, fino a soli 9–12 attributi, ma comporta un peggioramento evidente delle prestazioni. In particolare, l'F1-score diminuisce di circa il 3.5% rispetto alla configurazione originale, mentre ROC-AUC e PR-AUC mostrano una riduzione compresa tra l'1.6% e il 2.1%. Questo comportamento suggerisce che una selezione eccessivamente aggressiva elimini anche feature realmente informative, riducendo la capacità del modello di rappresentare le relazioni complesse presenti nel traffico di rete.

Dal punto di vista dell'efficienza computazionale, tutte le configurazioni ottenute mediante feature selection mostrano una riduzione del tempo medio di inferenza rispetto al modello originale. Nel caso della soglia *median* il tempo di predizione sull'intero test set diminuisce di circa il 13%, mentre le soglie più restrittive raggiungono riduzioni comprese tra il 16% e il 18%. È tuttavia interessante osservare come il miglioramento dell'efficienza non risulti proporzionale alla riduzione del numero di feature. Sebbene la configurazione selezionata riduca del 50% le variabili in ingresso, il tempo di inferenza diminuisce in misura più contenuta. Tale comportamento è coerente con il funzionamento di XGBoost, nel quale il costo computazionale della fase di predizione dipende principalmente dall'attraversamento degli alberi decisionali e dalla loro struttura, piuttosto che esclusivamente dal numero di feature disponibili.

Nel complesso, i risultati dimostrano che la feature selection costituisce una strategia efficace per semplificare il modello senza comprometterne le prestazioni predittive. Tra le soglie analizzate, la *median* rappresenta la soluzione più equilibrata, consentendo di dimezzare il numero di

feature mantenendo pressoché inalterata la capacità discriminativa del classificatore e ottenendo, al contempo, una riduzione del tempo di inferenza.

Le Tabelle 7, 8, 9 e 10 riassumono rispettivamente le feature selezionate, i risultati della cross validation comprensivi di media e deviazione standard, la configurazione finale del modello proposto e i risultati conseguiti sul test set.

Tabella 7: Feature selezionate dalla procedura di feature selection.

Feature	Feature	Feature
dpkts	sload	service_ftp-data
sbytes	dload	service_http
dbytes	sloss	service_smtp
dloss	swin	state_CON
tcprrt	synack	state_INT
ackdat	smean	proto_Other
dmean	response_body_len	proto_arp
ct_src_dport_ltm	is_sm_ips_ports	proto_ospf
proto_sctp	proto_udp	proto_unas
service_Unknown	service_ftp	

Tabella 8: Prestazioni medie del modello XGBoost (Cross-Validation)

Metrica	Media $\pm$ Dev. Standard
Accuracy	0.9231 $\pm$ 0.0016
F1-Score	0.9244 $\pm$ 0.0014
Precision	0.9002 $\pm$ 0.0038
Recall	0.9499 $\pm$ 0.0035
ROC-AUC	0.9831 $\pm$ 0.0001
PR-AUC	0.9814 $\pm$ 0.0005

Tabella 9: Configurazione finale del modello proposto.

Parametro	Valore
Algoritmo	XGBoost
Numero di feature	29
Soglia di selezione	median
Objective	binary:logistic
Evaluation metric	logloss
Tree method	hist
Random state	42
Scale positive weight	1.0202
Iperparametri ottimali	
Learning rate	0.1
Max depth	6
Number of estimators	200
Subsample	0.8
Colsample by tree	1.0

Tabella 10: Prestazioni finali del modello proposto.

# Feature	Accuracy	F1-score	ROC-AUC	PR-AUC	Inferenza (s)
29	0.853	0.864	0.975	0.977	0.118

Analisi della Qualità del Clustering

Come precedentemente descritto nel capitolo 3, entrambi gli scenari condividono lo stesso spazio delle feature (in entrambi i casi la colonna esclusa non fa parte dell’input di clustering, cambia solo la colonna esclusa), di conseguenza le metriche interne di clustering—Silhouette Score, Davies-Bouldin Index e Calinski-Harabasz Index—sono identiche tra i due scenari per ciascun valore di k ,

come confermato dalle tabelle 11 e 12. Questo è una conseguenza attesa del disegno sperimentale e indica che queste metriche interne sono una proprietà puramente legata alla partizione non supervisionata dello spazio delle feature, indipendente dal target utilizzato successivamente per la valutazione.

Il Silhouette Score, che misura il rapporto medio tra coesione intra-cluster e separazione inter-cluster per ciascun campione (con valori più vicini a 1 che indicano cluster compatti e ben separati), raggiunge il valore massimo osservato a $k = 2$ (0,408), diminuisce bruscamente a $k = 3$ (0,232), e poi aumenta gradualmente e in modo monotono fino a $k = 10$ (0,298), senza mai recuperare il valore ottenuto a $k = 2$. Il Davies-Bouldin Index, che quantifica la similarità media tra ciascun cluster e il suo omologo più simile (valori più bassi indicano cluster meglio separati), è minimizzato a $k = 7$ (1,190) ed è relativamente basso anche a $k = 2$ (1,216), mentre peggiora a $k = 3$ (1,535). Il Calinski-Harabasz Index, definito come il rapporto tra la dispersione inter-cluster e intra-cluster (valori più alti indicano una separazione più forte), è massimizzato a $k = 2$ (32.993,08) e diminuisce in modo monotono all'aumentare di k , raggiungendo 9.617,15 a $k = 10$.

Sulla base dei risultati osservati, le tre metriche interne non sono completamente concordanti: Calinski-Harabasz e, in misura minore, Silhouette favoriscono $k = 2$, mentre Davies-Bouldin favorisce $k = 7$. Questa discordanza indica che il dataset possiede una partizione geometrica dominante a due vie (che verosimilmente separa traffico ad alto/basso volume o benigno/anomalo) che massimizza la separazione globale, mentre una partizione più fine intorno a $k = 7$ produce cluster localmente più omogenei e meno sovrapposti, a scapito della dispersione inter-cluster complessiva. Nessun valore di k ottimizza simultaneamente tutti e tre gli indici interni, il che è coerente con la natura intrinsecamente continua e sovrapposta delle distribuzioni delle feature del traffico di rete.

Valutazione del Mapping Cluster-Classe

Poiché K-Means produce assegnazioni di cluster senza alcuna conoscenza delle etichette reali, è stata applicata una procedura di majority voting per tradurre la partizione non supervisionata in una predizione di classe: per ciascun cluster identificato sul training set, il valore più frequente del target di riferimento tra i campioni assegnati a quel cluster è stato selezionato come classe predetta del cluster, e questa mappatura è stata poi applicata, senza modifiche, ai cluster ottenuti

sul test set. Questa procedura non altera il clustering in sé; viene utilizzata esclusivamente come meccanismo di valutazione esterna per stimare in che misura la struttura geometrica scoperta da K-Means coincida con la struttura reale delle classi. Il suo vantaggio principale è che consente un confronto diretto e interpretabile tra i risultati del clustering e le metriche di classificazione; il suo limite principale è che può funzionare solo se ciascun cluster è effettivamente dominato da una singola classe, cosa che diventa sempre più difficile con l'aumentare del numero di categorie di attacco sottostanti, poiché un numero limitato di cluster non può rappresentare simultaneamente un gran numero di classi minoritarie.

Scenario di Classificazione Binaria

Nello scenario `excl_attack_cat`, l'Accuracy, la Precision, la Recall e l'F1-score mappati mostrano un chiaro miglioramento passando da $k = 2$ a $k = 4$: l'F1 sale da 0,422 a $k = 2$ a 0,749 a $k = 4$, e la Recall raggiunge 0,943 a $k = 4$, indicando che una partizione più fine dello spazio delle feature permette al mapping per majority voting di isolare meglio i cluster dominati dagli attacchi rispetto alla soluzione più grossolana a due cluster. Oltre $k = 4$, l'F1-score rimane relativamente stabile, raggiungendo il picco a $k = 8$ (0,751) e oscillando in un intervallo ristretto tra 0,741 e 0,751 fino a $k = 10$, mentre la Recall resta costantemente elevata (sopra 0,94). Questo pattern, in cui un aumento di k migliora l'accordo esterno di classificazione anche se il corrispondente Silhouette Score interno è più basso rispetto a $k = 2$, evidenzia una divergenza tra ciò che rende i cluster internamente compatti e ciò che li rende allineati con l'etichetta binaria reale.

Tabella 11: Risultati K-Means al variare di k , scenario binario (`excl_attack_cat`)

k	Silhouette	DB Index	CH Index	F1
2	0,408	1,216	32993,08	0,422
4	0,233	1,474	17460,02	0,749
7	0,253	1,190	10971,57	0,743
8	0,281	1,232	11368,82	0,751
10	0,298	1,353	9617,15	0,741

Sulla base dei risultati osservati, $k = 8$ rappresenta il miglior compromesso nello scenario

binario, poiché ottiene il più alto F1-score mappato (0,751) tra tutte le configurazioni testate, mantenendo una Recall superiore a 0,97, al costo di una riduzione solo moderata del Silhouette Score rispetto a $k = 2$. La configurazione $k = 2$, nonostante massimizzi le metriche di separazione interna, produce l'accordo esterno più debole con l'etichetta binaria, indicando che la suddivisione a due vie geometricamente più distinta dello spazio delle feature non corrisponde bene al confine Normal/Attack.

Scenario di Classificazione Multiclasse

Nello scenario `excl_label`, tutte le metriche esterne mappate sono sostanzialmente più basse rispetto al caso binario: l'Accuracy mappata varia tra 0,345 e 0,516, l'F1 macro rimane sotto 0,13 per ogni valore di k testato, e l'F1 weighted varia tra 0,303 e 0,462. Il divario tra F1 macro e weighted è costantemente ampio (ad esempio, a $k = 9$, l'F1 macro è 0,129 contro un F1 weighted di 0,462), il che indica che le predizioni mappate ottengono buoni risultati solo sulla classe maggioritaria (Normal), fallendo quasi completamente nel recuperare le categorie di attacco minoritarie, poiché la media macro pesa ogni classe in modo uguale mentre la media weighted è dominata dalla classe più frequente. Il miglior F1 macro (0,129) si ottiene a $k = 9$, così come il miglior F1 weighted (0,462), mentre la miglior Accuracy mappata (0,516) si verifica a $k = 2$, dove entrambi i cluster sono stati mappati su "Normal", collassando di fatto il classificatore in un predittore banale della classe maggioritaria.

Tabella 12: Risultati K-Means al variare di k , scenario multiclasse (`excl_label`)

k	Silhouette	DB Index	CH Index	F1 (macro/weighted)
2	0,408	1,216	32993,08	0,068 / 0,351
7	0,253	1,190	10971,57	0,119 / 0,448
9	0,291	1,232	10480,87	0,129 / 0,462
10	0,298	1,353	9617,15	0,120 / 0,376

Sulla base dei risultati osservati, nessuna configurazione nello scenario multiclasse raggiunge un F1 macro superiore a 0,13, il che indica che il clustering non supervisionato K-Means, anche consentendo al numero di cluster di crescere fino a 10, non è in grado di recuperare in modo

significativo la struttura fine delle categorie di attacco tramite il solo majority voting. Mapping dei cluster come {0 : 'Exploits', 1 : 'Normal', 2 : 'Fuzzers', 3 : 'Normal'} a $k = 4$ mostrano che diversi cluster sono ancora dominati da "Normal", confermando che la classe benigna numericamente prevalente sovrasta l'assegnazione a majority voting nella maggior parte delle partizioni.

Visualizzazione dei Cluster tramite PCA

La Principal Component Analysis è stata applicata esclusivamente per scopi di visualizzazione bidimensionale, proiettando le feature standardizzate del test set sulle prime due componenti principali; la PCA non è stata utilizzata come input per l'algoritmo K-Means e pertanto non ha alcuna influenza sul risultato del clustering. Per ciascuna configurazione, il codice produce due scatter plot complementari sulla stessa proiezione PCA: uno colorato in base all'assegnazione di cluster di K-Means e uno colorato in base all'etichetta reale (Normal/Attack per lo scenario binario, o categoria di attacco per lo scenario multiclasse). Non sono stati forniti file immagine né dati dei pixel dei grafici nei risultati disponibili, pertanto non è possibile derivare una descrizione quantitativa o qualitativa dell'effettiva separazione visiva osservata in questi grafici PCA dalle evidenze disponibili; è possibile riportare solo le metriche associate ai corrispondenti risultati numerici nella Tabella 11 e nella Tabella 12.

Discussione

Apprendimento supervisionato

L'analisi metodologica condotta fa emergere considerazioni critiche riguardanti l'implementazione di sistemi di Network Intrusion Detection (NIDS) basati su Machine Learning in scenari operativi reali, quali i Security Operations Center (SOC). Rispondendo alle questioni di ricerca poste alla base di questo studio (in particolare in merito al divario tra accuratezza teorica e applicabilità pratica), si evince che il problema principale in produzione non è unicamente la capacità predittiva del modello in sé, ma l'impatto dei suoi output sugli analisti umani. La decisione di privilegiare metriche quali la *Recall* e la *Precision* riflette la necessità di limitare la cosiddetta *Alert Fatigue*. Modelli teoricamente accurati ma con bassi livelli di precisione inonderebbero il SOC di falsi

positivi, rendendo l'intero sistema inaffidabile e innescando criticità operative insostenibili.

In aggiunta, in risposta agli interrogativi riguardanti i bias metodologici, l'architettura della pipeline qui proposta offre una forte garanzia contro il *Data Leakage*, un errore sistematico e distruttivo presente in molta letteratura pregressa. Frequenti implementazioni non corrette dello splitting dei dati o l'inclusione di feature contestuali, come indirizzi IP sorgente/destinazione o identificatori sequenziali, portano tipicamente il modello ad apprendere la topologia specifica e le anomalie sintetiche della rete di test, generando *spurious correlations* anziché il pattern comportamentale generalizzabile dell'attacco. Concentrandosi unicamente sulle proprietà semantiche di flusso e sul contenuto statistico intrinseco (pacchetti, byte, jitter, latenza), lo studio garantisce una reale capacità di generalizzazione del modello verso reti e infrastrutture informatiche eterogenee, superando le limitazioni tipiche dei framework chiusi.

I risultati sperimentali ottenuti confermano in larga misura le evidenze emerse dalla letteratura analizzata e consentono di rispondere direttamente alla questione di ricerca relativa alla selezione dell'algoritmo ottimale per i dati di rete. Nel complesso, gli algoritmi supervisionati basati su ensemble di alberi decisionali hanno mostrato prestazioni nettamente superiori rispetto ai modelli lineari. Ciò dimostra strutturalmente che il problema della classificazione del traffico di rete nel dataset UNSW-NB15 è caratterizzato da relazioni complesse, eterogenee e fortemente non lineari tra le variabili descrittive, che sfuggono alle proiezioni geometriche continue tipiche dei classificatori lineari.

Tra tutti i classificatori valutati, XGBoost ha ottenuto le migliori prestazioni complessive, seguito da Random Forest ed Extra Trees. Tale risultato risponde alle indagini della letteratura recente, che identifica gli algoritmi di gradient boosting come la soluzione d'elezione per la classificazione di dati tabellari ad alta dimensionalità. La capacità di XGBoost di costruire progressivamente alberi che correggono, tramite regolarizzazione integrata, i residui d'errore dei modelli precedenti, consente di apprendere relazioni complesse tra le feature mantenendo un'elevata capacità di generalizzazione e mitigando il rischio di overfitting intrinseco nel traffico di rete.

Le prestazioni ottenute da Random Forest ed Extra Trees confermano inoltre l'efficacia delle tecniche di bagging nel contesto dell'Intrusion Detection, affrontando il problema della varianza del dato. Sebbene i due modelli risultino leggermente inferiori a XGBoost in termini di efficacia predittiva globale, entrambi hanno mostrato un'elevata robustezza ai rumori e una notevole capacità

di individuare il traffico malevolo, raggiungendo valori di *Recall* superiori al 97%. Extra Trees, in particolare, ha privilegiato una sensibilità ancora maggiore nei confronti degli attacchi, a fronte di un lieve incremento del numero di falsi positivi, evidenziando il classico e ineliminabile compromesso tra *Recall* e *Precision* nella rilevazione di anomalie minoritarie.

I risultati dei modelli lineari mostrano per contro i limiti intrinseci della loro capacità rappresentativa. Logistic Regression e Linear SVM hanno sì ottenuto valori di *Recall* elevati, dimostrando una discreta propensione a identificare le intrusioni, ma hanno evidenziato una *Precision* significativamente inferiore rispetto ai modelli ensemble. Questo comportamento suggerisce un deficit strutturale nel mappare la complessità dello spazio delle caratteristiche del dataset: le frontiere decisionali lineari risultano insufficienti per gestire le interazioni condizionali tra i protocolli e sono intrinsecamente vulnerabili agli outlier numerici (ad esempio, attacchi volumetrici), che invece vengono isolati naturalmente dagli split ortogonali degli alberi decisionali.

Di fondamentale importanza per rispondere alle questioni sui framework di valutazione è l'analisi delle curve ROC e Precision-Recall. L'approccio adottato supera la fallacia del tasso di base (*Base Rate Fallacy*) legata al massiccio sbilanciamento dei flussi di rete. Sebbene gli elevati valori di ROC-AUC ottenuti da XGBoost, Random Forest ed Extra Trees confermino la superiorità dei modelli ensemble, è la coerenza con i valori della curva PR-AUC a validare scientificamente l'eccellente capacità discriminativa sulle classi minoritarie. L'impiego della PR-AUC e dell'F1-score assicura che le prestazioni misurate non siano un artefatto matematico gonfiato dall'elevato numero di veri negativi (traffico benigno), ma riflettano un'autentica robustezza del modello indipendente dalla soglia operativa scelta.

Sul fronte dell'interpretabilità e della selezione delle caratteristiche, l'analisi dell'importanza delle feature ha evidenziato una notevole convergenza tra i principali algoritmi ensemble. Variabili semantiche come *dload*, *tcprtt*, *synack* e *ackdat* risultano sistematicamente tra le più informative. Questo risultato è cruciale sotto due aspetti: in primo luogo, indica che il volume di traffico e le latenze legate ai *three-way handshake* del protocollo TCP sono indicatori universali per il riconoscimento delle intrusioni; in secondo luogo, dimostra che mantenere le feature nel loro dominio originale (feature selection) piuttosto che proiettarle in spazi latenti oscuri (come avverrebbe tramite PCA o modelli di Deep Learning) preserva l'interpretabilità (*Explainable AI*). La coerenza tra modelli accresce la fiducia operativa, suggerendo che le logiche decisionali estratte siano fisicamente

fondate e non mere memorizzazioni statistiche.

Dal punto di vista metodologico, il rigoroso protocollo sperimentale ha fornito una risposta tangibile alle vulnerabilità dei flussi di validazione. La rigida separazione tra training e test set, la definizione del preprocessing esclusivamente sui dati di addestramento e l'ottimizzazione degli iperparametri mediante Grid Search con validazione incrociata hanno sterilizzato il rischio di data leakage. Il fatto che le prestazioni in cross-validation risultino solo marginalmente superiori a quelle del test set indipendente attesta che il sistema ha raggiunto un equilibrio ottimale tra bias e varianza, tollerando un livello di overfitting fisiologico ma garantendo una solida trasferibilità del modello.

Nonostante i risultati ottenuti siano complessivamente validi e ben supportati, l'interpretazione critica degli stessi richiede l'esposizione di alcune limitazioni oggettive. In primo luogo, la sperimentazione è stata vincolata all'uso del dataset UNSW-NB15: pur rappresentando un netto passo avanti rispetto ai dataset legacy, esso fotografa una topologia di rete statica e non può abbracciare l'infinita varianza e le dinamiche di *concept drift* del traffico in continua evoluzione reale. In secondo luogo, l'analisi si è limitata alla classificazione binaria. Benché essenziale per discriminare il traffico benigno da quello malevolo, tale approccio omette la classificazione multiclasse delle specifiche famiglie di malware, un'informazione spesso necessaria agli analisti SOC per stabilire le priorità di mitigazione.

Apprendimento non supervisionato

Le evidenze sperimentali disponibili indicano che K-Means è in grado di recuperare una distinzione binaria grossolana tra traffico benigno e malevolo con efficacia moderata per configurazioni con $k \geq 4$, ma la sua capacità di recuperare la struttura fine delle categorie di attacco è molto limitata nello scenario multiclasse. Questa discrepanza è attesa data la natura intrinsecamente non supervisionata dell'algoritmo: K-Means partiziona lo spazio delle feature esclusivamente sulla base della distanza euclidea dai centroidi, senza alcun meccanismo per tenere conto dello sbilanciamento delle classi o del fatto che le categorie di attacco minoritarie possano occupare solo una regione piccola, potenzialmente non convessa, dello spazio delle feature che non corrisponde a un cluster naturale basato sulla densità. I classificatori supervisionati, al contrario, sono ottimizzati direttamente rispetto al confine di decisione etichettato e possono quindi apprendere feature discriminative per le classi minoritarie anche quando queste non formano un cluster geometrico distinto, il che spiega

il notevole divario tra i risultati multiclasse qui ottenuti e quanto tipicamente ci si aspetterebbe da un classificatore multiclasse supervisionato sulle stesse feature.

La scelta di k ha un effetto misurabile ma non banale sia sulle metriche interne che su quelle esterne, e le due non sono sempre allineate: metriche interne come Calinski-Harabasz favoriscono costantemente valori più piccoli di k , mentre metriche esterne guidate dalle etichette come l’F1 mappato favoriscono valori intermedi di k sia nello scenario binario sia in quello multiclasse. Questa divergenza illustra un limite fondamentale nell’utilizzo di criteri puramente interni di clustering per selezionare il numero di cluster quando l’obiettivo finale è il recupero delle classi: una partizione internamente ottimale non è garantita essere esternamente significativa rispetto a uno specifico schema di etichettatura. Inoltre, il mapping per majority voting utilizzato per la valutazione esterna converte K-Means in un classificatore de facto solo a posteriori, il che significa che le sue prestazioni non dovrebbero essere interpretate come evidenza di una genuina separabilità delle classi in modo non supervisionato, ma piuttosto come una misura a posteriori di quanto una partizione geometrica arbitraria si allinei con la distribuzione delle etichette.

Nel complesso, sulla base dei risultati osservati, K-Means mostra una capacità limitata ma non trascurabile di rilevare differenze strutturali ampie tra traffico normale e traffico di attacco, il che lo rende potenzialmente utile come strumento leggero e senza etichette di pre-filtraggio o esplorazione all’interno di una pipeline NIDS, ma non è adatto come rilevatore autonomo per la classificazione a grana fine delle categorie di attacco, date le prestazioni complessivamente basse ottenute su tutto l’intervallo esplorato di k . Per lo scenario binario, $k = 8$ è selezionato come configurazione con le prestazioni migliori sulla base del più alto F1-score mappato ottenuto tra tutti i valori testati, combinato con un Davies-Bouldin Index prossimo al minimo osservato; per lo scenario multiclasse, $k = 9$ è selezionato sulla base dei migliori valori di F1 macro e F1 weighted ottenuti congiuntamente, nonostante le prestazioni assolute complessivamente basse di ogni configurazione testata.

Conclusione

Il presente lavoro ha sviluppato e validato una pipeline metodologica completa per la preparazione, la modellazione e la valutazione di sistemi di Network Intrusion Detection basati sul dataset

UNSW-NB15. A differenza di molti studi che concentrano l'attenzione esclusivamente sulla scelta dell'algoritmo di classificazione, questo lavoro ha affrontato l'intero processo di sviluppo del modello, evidenziando come la qualità delle prestazioni finali dipenda in misura determinante dalla correttezza del protocollo sperimentale adottato.

L'analisi esplorativa dei dati ha evidenziato numerose criticità tipiche dei dataset destinati all'Intrusion Detection, tra cui la presenza di un'elevata percentuale di osservazioni duplicate nel training set, distribuzioni fortemente asimmetriche, numerosi outlier, variabili categoriche ad alta cardinalità e relazioni di ridondanza tra alcune feature. Tali evidenze hanno guidato la progettazione di una pipeline di preprocessing rigorosa, basata sulla rimozione delle sole ridondanze informative, sulla trasformazione logaritmica delle variabili altamente asimmetriche, sulla codifica appropriata delle variabili categoriche e sulla completa eliminazione di ogni possibile forma di data leakage tra training e test set. In particolare, la rimozione dal test set delle osservazioni identiche presenti nel training ha consentito di ottenere una valutazione realmente indipendente delle prestazioni, sacrificando la perfetta aderenza al benchmark ufficiale in favore di una maggiore validità metodologica.

Nel complesso, i risultati ottenuti dimostrano che il successo di un sistema di Intrusion Detection non dipende esclusivamente dall'impiego di algoritmi avanzati, ma soprattutto dalla qualità del processo di preparazione dei dati, dalla corretta progettazione del protocollo sperimentale e dalla scelta di metriche di valutazione coerenti con gli obiettivi operativi del dominio della cybersecurity. Una pipeline metodologicamente rigorosa consente infatti di ottenere modelli più robusti, maggiormente interpretabili e capaci di generalizzare su dati non osservati, riducendo il rischio di sovrastimare artificialmente le prestazioni attraverso procedure sperimentali non corrette.

Il lavoro presenta tuttavia alcune limitazioni. La sperimentazione è stata condotta esclusivamente sul dataset UNSW-NB15, che, pur rappresentando uno dei benchmark più completi disponibili, non è in grado di riprodurre la continua evoluzione del traffico di rete reale e dei moderni scenari di attacco. Inoltre, l'analisi si è concentrata principalmente sul problema di classificazione binaria, rimandando a studi successivi la classificazione delle singole famiglie di attacco, caratterizzate da un marcato sbilanciamento delle classi.

Gli sviluppi futuri potranno estendere il presente lavoro in diverse direzioni. In primo luogo, sarà opportuno affrontare il problema multiclasse mediante tecniche specifiche per la gestione delle

classi rare, quali pesi di classe, strategie di campionamento e funzioni di costo bilanciate. Ulteriori sviluppi riguarderanno l'introduzione di tecniche di Explainable Artificial Intelligence, come SHAP e LIME, per migliorare l'interpretabilità delle decisioni dei modelli, nonché la valutazione di approcci di apprendimento continuo in grado di adattarsi al fenomeno del concept drift tipico degli ambienti di rete reali. Infine, la validazione della pipeline su dataset differenti e su traffico acquisito in contesti operativi consentirà di verificare ulteriormente la capacità di generalizzazione del framework proposto e la sua effettiva applicabilità nei moderni sistemi di difesa informatica.

A Ambiente sperimentale

Tutti gli esperimenti presentati in questo lavoro sono stati eseguiti utilizzando Google Colab come ambiente di sviluppo, collegato a un runtime locale in esecuzione su un personal computer con sistema operativo Windows, utilizzando Python 3.12.13 e le librerie riportate nella Tabella 13.

Tabella 13: Versioni del software utilizzato negli esperimenti.

Software	Versione
Python	3.12.13
NumPy	2.0.2
pandas	2.2.2
scikit-learn	1.6.1
SciPy	1.16.3
XGBoost	3.2.0
matplotlib	3.10.0
seaborn	0.13.2
missingno	0.5.2
joblib	1.5.3

B Ambiente di esecuzione

L'addestramento e la valutazione dei modelli sono stati eseguiti sul processore del computer locale mediante il runtime locale di Google Colab. Per gli algoritmi che supportano l'elaborazione parallela è stata utilizzata l'opzione `n_jobs=6`. I tempi di addestramento riportati nel capitolo dei risultati dipendono pertanto dall'hardware impiegato e hanno valore puramente indicativo.

Riferimenti Bibliografici e Note

1. N. Moustafa, J. Slay, UNSW-NB15: a comprehensive data set for network intrusion detection systems (UNSW-NB15 network data set), in *Military Communications and Information Systems Conference (MilCIS)* (IEEE) (2015).
2. N. Moustafa, J. Slay, The evaluation of Network Anomaly Detection Systems: Statistical analysis of the UNSW-NB15 dataset and the comparison with the KDD99 dataset. *Information Security Journal: A Global Perspective* pp. 1–14 (2016).
3. N. Moustafa, *et al.*, Novel geometric area analysis technique for anomaly detection using trapezoidal area estimation on large-scale networks. *IEEE Transactions on Big Data* (2017).
4. N. Moustafa, *et al.*, Big data analytics for intrusion detection system: statistical decision-making using finite dirichlet mixture models, in *Data Analytics and Decision Support for Cybersecurity* (Springer, Cham), pp. 127–156 (2017).
5. M. Sarhan, S. Layeghy, N. Moustafa, M. Portmann, NetFlow Datasets for Machine Learning-Based Network Intrusion Detection Systems, in *Big Data Technologies and Applications: 10th EAI International Conference, BDTA 2020* (Springer Nature) (2020), p. 117.